SRMECOTECH

# Database Schema Specification

PostgreSQL database gps_tracking (Flyway V1 to V11), with the Redis key space, Kafka topics and state stores of the GPS Processing Layer

| | |
|---|---|
| **Document identifier** | SRM-GPS-DBS-001 |
| **Version** | 0.11 |
| **Status** | Draft for review (not yet baselined) |
| **Date** | 3 November 2026 |
| **Prepared by** | Sanjoy Chattopadhay, SRMECOTECH |
| **Classification** | Internal |
| **Governing standard** | ISO/IEC/IEEE 29148:2018 (data requirements) |

*Companion to SRM-GPS-SRS-001. The schema described here is the one created by the Flyway migrations V1 to V11 in gps-application/src/main/resources/db/migration; if the two ever differ, the migrations are authoritative and this document shall be corrected.*

# Document Control

This document is placed under configuration control. Changes are made only through the change-control procedure in the companion document “SRS Rules and Regulations” and are recorded in the revision history below. A version number ending in “.0” denotes a baselined (approved) issue; any other number denotes a working draft.

***Table 1:*** *Revision history*

| Version | Date | Author | Description of change |
|---|---|---|---|
| 0.1 | 19 Oct 2026 | S. Chattopadhay | Tables listed in README section 11 (V1 to V4). |
| 0.9 | 29 Oct 2026 | S. Chattopadhay | First formal specification: data dictionary for all 15 tables (V1 to V8), constraints, indexes, relationships, lifecycle, Redis, Kafka and state stores, full DDL. |
| 0.11 | 3 Nov 2026 | S. Chattopadhay | V9 (transport management: sign-in, places, shipments), V10 (geofencing: place as the geofence master, geofence_visit) and V11 (route plans and stops): eight new tables, second ER figure, new value sets, queries and idempotency keys. |

***Table 2:*** *Review and approval*

| Role | Name | Signature | Date |
|---|---|---|---|
| Technical lead / architect | | | |
| Database administrator | | | |
| Quality assurance lead | | | |

# Table of Contents

Document Control ..... 2
1 Introduction ..... 5
1.1 Purpose ..... 5
1.2 Scope ..... 5
1.3 References ..... 5
1.4 Notation ..... 5
2 Database Overview ..... 6
2.1 Subject Areas ..... 6
3 Design Principles and Naming Conventions ..... 9
3.1 Design Principles ..... 9
3.2 Naming Conventions ..... 9
4 Data Dictionary ..... 10
4.1 Table tenant ..... 10
4.2 Table vehicle ..... 10
4.3 Table device ..... 11
4.4 Table device_assignment ..... 11
4.5 Table gps_position ..... 12
4.6 Table vehicle_motion_state ..... 13
4.7 Table gps_trip ..... 14
4.8 Table gps_stop ..... 15
4.9 Table gps_trip_late_reading ..... 16
4.10 Table vehicle_event ..... 17
4.11 Table dead_letter ..... 18
4.12 Table rejected_frame ..... 19
4.13 Table connection_setting ..... 20
4.14 Table vehicle_day_summary ..... 20
4.15 Table report_setting ..... 21
4.16 Tables added by V9 to V11 ..... 22
4.17 Table app_user ..... 22
4.18 Table auth_key ..... 23
4.19 Table place ..... 23
4.20 Table shipment_import ..... 24
4.21 Table shipment ..... 25
4.22 Table geofence_visit ..... 26
4.23 Table route_plan ..... 27

4.24 Table route_stop....28
5 Relationships and Referential Integrity....30
6 Value Sets....31
7 Index Strategy and Main Queries....32
8 Data Integrity, Idempotency and Concurrency....33
9 Data Lifecycle: Retention, Clearing, Backup and Recovery....34
9.1 Retention....34
9.2 Clear Database and Demo Reset....34
9.3 Backup and Recovery....34
9.4 Capacity Estimate....34
10 Migration History and Change Rules....36
11 Non-Relational Data Stores....37
11.1 Redis Key Space....37
11.2 Kafka Topics....37
11.3 Kafka Streams State Stores....38
12 External Source: smart_truck (Read-Only)....39
13 Open Issues....40
14 Appendices....41
14.1 Appendix A: Complete DDL....41
14.1.1 V1__create_master_data_tables.sql....41
14.1.2 V2__create_gps_position_table.sql....41
14.1.3 V3__create_vehicle_motion_state_table.sql....42
14.1.4 V4__create_gps_trip_and_gps_stop_tables.sql....42
14.1.5 V5__distance_details_and_vehicle_events.sql....43
14.1.6 V6__dead_letter_and_rejected_frame_archive.sql....44
14.1.7 V7__connection_setting.sql....44
14.1.8 V8__vehicle_day_summary_and_report_setting.sql....45
14.1.9 V9__transport_management.sql....45
14.1.10 V10__geofencing.sql....49
14.1.11 V11__route_plans.sql....53

# 1 Introduction

## 1.1 Purpose

This document specifies the persistent data of the GPS Processing Layer: the PostgreSQL database gps_tracking in full, and the Redis key space, Kafka topics and Kafka Streams state stores that hold the rest of the platform's state. It is the physical counterpart of Sections 3.2 and 6 of the Software Requirements Specification SRM-GPS-SRS-001 and satisfies its data requirements DR-001 to DR-010.

## 1.2 Scope

- Twenty-three PostgreSQL tables created by Flyway migrations V1 to V11: master data, telemetry, trip detection, events, reporting, archives, configuration, sign-in, shipments, geofencing and route plans.
- Constraints, indexes, relationships, value sets, write patterns, idempotency keys, retention and recovery.
- Redis keys (live state, configuration, counters), Kafka topics and Kafka Streams state stores.
- The read-only legacy source smart_truck, as far as this platform reads it.

## 1.3 References

***Table 3:*** *Referenced documents*

| Ref. | Document |
|---|---|
| [S1] | SRM-GPS-SRS-001, Software Requirements Specification: GPS Processing Layer. |
| [S2] | SRM-GPS-SRS-RUL-001, SRS Rules and Regulations. |
| [S3] | gps-application/src/main/resources/db/migration/V1 to V11 (authoritative DDL). |
| [S4] | README.md sections 11, 12, 14 and 15; docs/DATA_SAFETY_GUIDE.md; docs/REPORTS_GUIDE.md; docs/TMS_GUIDE.md; docs/GEOFENCING_GUIDE.md; docs/ROUTE_DEVIATION_ETA_DESIGN.md. |
| [S5] | PostgreSQL 14 documentation: data types, constraints (EXCLUDE with btree_gist), indexes. |

## 1.4 Notation

***Table 4:*** *Notation used in this document*

| Symbol | Meaning |
|---|---|
| PK | Primary key |
| UK | Unique key (UNIQUE constraint) |
| FK | Foreign key |
| CK | CHECK constraint |
| EX | Exclusion constraint (EXCLUDE USING gist) |
| IX | Secondary index |
| Null = N | Column is NOT NULL |
| Logical reference | A column that names a row of another table (for example imei) without a foreign key; integrity is kept by the application (Section 5). |
| timestamptz | timestamp with time zone; every value is stored and compared as an absolute instant (UTC). |

# 2 Database Overview

***Table 5:*** *Database facts*

| Item | Value |
|---|---|
| **DBMS** | PostgreSQL 14 (Docker image postgres:14; native 14.22) |
| **Database / schema** | gps_tracking / public |
| **Application user** | gps_app (password from local-secrets.properties or .env; never in version control) |
| **Extensions** | btree_gist (needed for the exclusion constraint on device_assignment) |
| **Schema management** | Flyway, migrations V1 to V11; Hibernate ddl-auto = validate (it never changes the schema) |
| **Time zone** | JDBC session time zone UTC (hibernate.jdbc.time_zone); all timestamps are timestamptz |
| **Character set** | UTF-8 |
| **Connection** | jdbc:postgresql://localhost:5432/gps_tracking (Docker: postgres:5432, host port 5433) |

## 2.1 Subject Areas

***Table 6:*** *Tables by subject area*

| Subject area | Tables | Written by |
|---|---|---|
| Master data | tenant, vehicle, device, device_assignment | SimulationMasterDataSeeder (future: master-data admin API) |
| Telemetry | gps_position | Consumer group gps-position-storage-service (or the direct flow) |
| Trip detection | vehicle_motion_state, gps_trip, gps_stop, gps_trip_late_reading | Consumer group gps-trip-service |
| Events | vehicle_event | Speed-violation, vehicle-state, stoppage and ignition consumers |
| Reporting | vehicle_day_summary, report_setting | DaySummaryJob; Reports → Cost assumptions |
| Archives | dead_letter, rejected_frame | gps-dead-letter-archive; gps-rejected-frame-logger |
| Configuration | connection_setting | Developer → Connections |
| Sign-in (V9) | app_user, auth_key; tenant.kind | AuthController, PortalAdminController |
| Shipments (V9) | shipment, shipment_import, place | ShipmentImportService (Excel / CSV / tta_trips), module shipment-tracking |
| Geofencing (V10) | place (geofence master), geofence_visit | Live geofences page, module geofence |
| Route plans (V11) | route_plan, route_stop | RoutePlanService, modules route-deviation and waypoints |

Figure 1 (next page) shows the tables of V1 to V8 and their relationships; Figure 2 those added by V9 to V11.

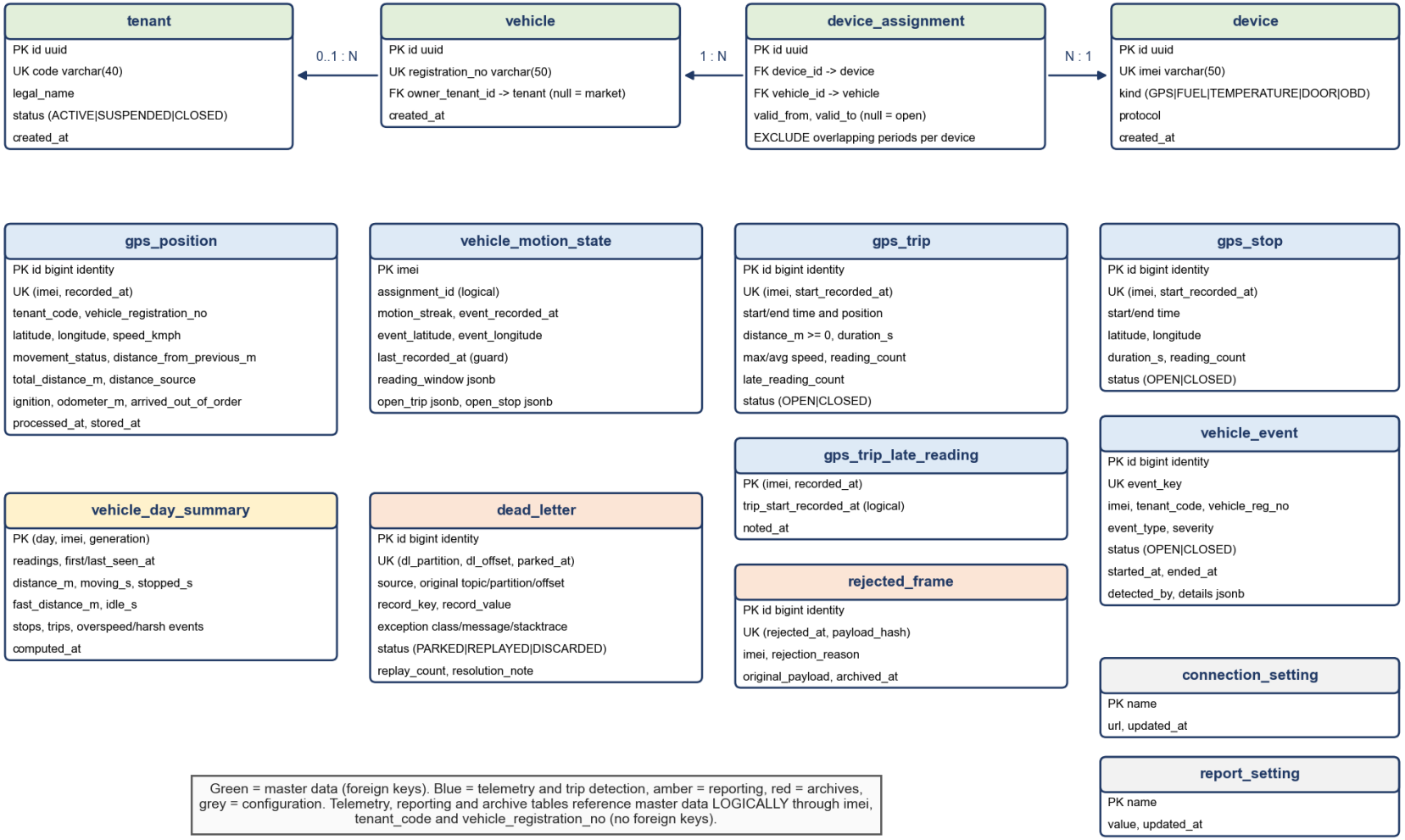



***Figure 1:*** *Entity-relationship overview of gps_tracking*

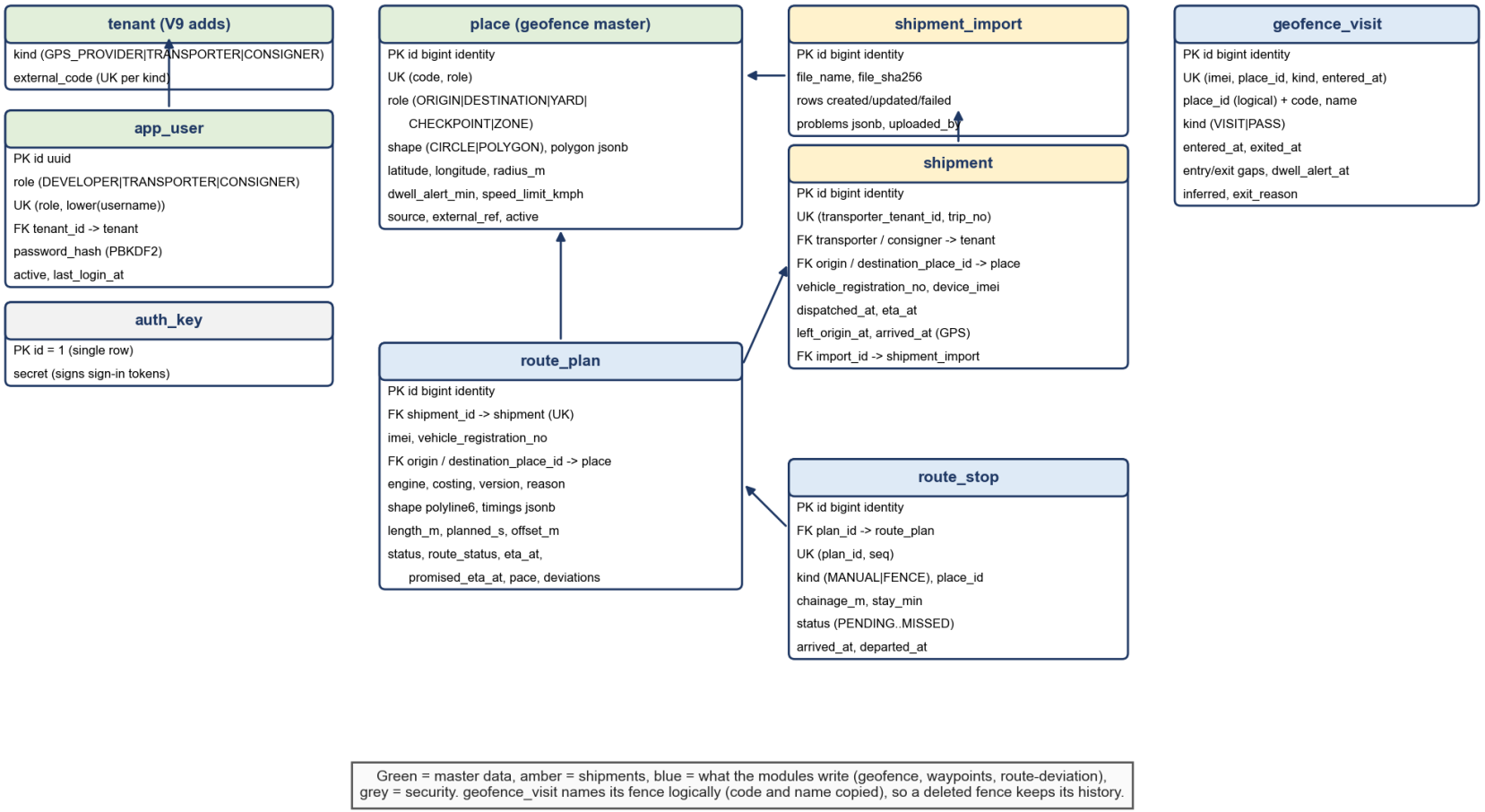



***Figure 2:*** *Tables added by V9 to V11: sign-in, shipments, geofencing, route plans*

# 3 Design Principles and Naming Conventions

## 3.1 Design Principles

***Table 7:*** *Design decisions*

| Decision | Rationale |
|---|---|
| Master data uses uuid keys and foreign keys. | Few rows, changed rarely, referenced by other master data: the database guarantees integrity. |
| Telemetry uses bigint identity keys and logical references (imei, tenant_code, vehicle_registration_no). | Written at high rate; readable without joins; must survive master-data changes; planned to move to a time-series store. |
| Device assignments are periods, never a “current vehicle” column. | A late reading must be matched to the vehicle the device was in at the reading's time (BR-004). |
| Every consumer-written table has a natural idempotency key. | Kafka delivers at least once; a redelivered record must not create a second row (Section 8). |
| Trip state is updated with a guard on last_recorded_at. | A repeated or late reading updates 0 rows, so the whole transaction is rolled back. |
| Value sets are CHECK constraints, not lookup tables. | Small, stable sets; the check documents the allowed values next to the column. |
| Archives are never deleted; status changes instead. | Every record that did not become a position stays traceable (BR-006). |
| Reports read a summary table. | One row per vehicle per day keeps month reports small whatever the position volume. |

## 3.2 Naming Conventions

- Tables: singular, lower snake_case (gps_position, not GpsPositions).
- Time columns end in `_at` and are timestamptz; a date-only column is named `day`.
- Units are suffixes: `_m` metres, `_s` seconds, `_kmph` km/h, `_meters` in older columns of the same meaning.
- Constraints: <table>_<rule> (gps_trip_distance_not_negative); indexes: <table>_<columns or purpose>.
- Status columns hold upper-case words (OPEN, CLOSED, PARKED …).

# 4 Data Dictionary

For each table: purpose, writer and readers, write pattern, columns, constraints and indexes. Column order follows the migration that created it; columns added later (V5) are listed at the end.

## 4.1 Table tenant

| Item | Value |
|---|---|
| **Purpose** | A customer company on the platform. |
| **Created in** | V1__create_master_data_tables.sql |
| **Written by / read by** | SimulationMasterDataSeeder / DeviceVehicleDirectory, reports filters |
| **Expected size** | Tens of rows |

***Table 8:*** *tenant columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | uuid | N | | Surrogate key, generated by the application. |
| 2 | **code** | varchar(40) | N | | Short lower-case code stamped on every position, never re-used (BR-003). |
| 3 | **legal_name** | varchar(255) | N | | Registered company name. |
| 4 | **status** | varchar(20) | N | 'ACTIVE' | ACTIVE, SUSPENDED or CLOSED. |
| 5 | **created_at** | timestamptz | N | now() | Creation time. |

***Table 9:*** *tenant constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| tenant_pkey | PK | (id) |
| tenant_code_key | UK | (code): one code per tenant, platform-wide. |
| tenant_code_format | CK | code ~ '^[a-z0-9][a-z0-9_-]*$' |
| tenant_status_values | CK | status IN ('ACTIVE','SUSPENDED','CLOSED') |

## 4.2 Table vehicle

| Item | Value |
|---|---|
| **Purpose** | A truck, identified by its registration number. |
| **Created in** | V1 |
| **Written by / read by** | SimulationMasterDataSeeder / DeviceVehicleDirectory |
| **Expected size** | Hundreds to thousands of rows |

***Table 10:*** *vehicle columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | uuid | N | | Surrogate key. |
| 2 | **registration_no** | varchar(50) | N | | Registration plate, unique across the platform (BR-002). |
| 3 | **owner_tenant_id** | uuid | Y | | Owning tenant; null means a market vehicle. |
| 4 | **created_at** | timestamptz | N | now() | Creation time. |

*Table 11: vehicle constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| vehicle_pkey | PK | (id) |
| vehicle_registration_no_key | UK | (registration_no) |
| vehicle_owner_tenant_id_fkey | FK | owner_tenant_id → tenant(id); default NO ACTION on delete and update. |

## 4.3 Table device

| Item | Value |
|---|---|
| **Purpose** | A tracking device, known only by its IMEI. |
| **Created in** | V1 |
| **Written by / read by** | SimulationMasterDataSeeder / DeviceVehicleDirectory |
| **Expected size** | Hundreds to thousands of rows |

*Table 12: device columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | uuid | N | | Surrogate key. |
| 2 | **imei** | varchar(50) | N | | Device identity; the Kafka record key of all its messages. |
| 3 | **kind** | varchar(20) | N | 'GPS' | GPS, FUEL, TEMPERATURE, DOOR or OBD. |
| 4 | **protocol** | varchar(50) | Y | | Device protocol name, for the future parser. |
| 5 | **created_at** | timestamptz | N | now() | Creation time. |

*Table 13: device constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| device_pkey | PK | (id) |
| device_imei_key | UK | (imei) |
| device_kind_values | CK | kind IN ('GPS','FUEL','TEMPERATURE','DOOR','OBD') |

## 4.4 Table device_assignment

| Item | Value |
|---|---|
| **Purpose** | The period during which a device is installed in a vehicle. |
| **Created in** | V1 |
| **Written by / read by** | SimulationMasterDataSeeder / DeviceVehicleDirectory (cached per period) |
| **Rule** | A device is in at most one vehicle at any instant (BR-001). The period is half-open: [valid_from, valid_to). |

*Table 14: device_assignment columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | uuid | N | | Surrogate key; also stored in vehicle_motion_state.assignment_id. |

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 2 | **device_id** | uuid | N | | The device. |
| 3 | **vehicle_id** | uuid | N | | The vehicle. |
| 4 | **valid_from** | timestamptz | N | | Start of the installation (inclusive). |
| 5 | **valid_to** | timestamptz | Y | | End (exclusive); null while still installed. |

***Table 15:*** *device_assignment constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| device_assignment_pkey | PK | (id) |
| device_assignment_device_id_fkey | FK | device_id → device(id) |
| device_assignment_vehicle_id_fkey | FK | vehicle_id → vehicle(id) |
| device_assignment_period_is_valid | CK | valid_to IS NULL OR valid_to > valid_from |
| device_assignment_one_vehicle_per_device_at_a_time | EX | EXCLUDE USING gist (device_id WITH =, tstzrange(valid_from, valid_to) WITH &&): no two periods of one device overlap. |

***Table 16:*** *device_assignment indexes*

| Index | Columns | Serves |
|---|---|---|
| device_assignment_vehicle | (vehicle_id) | Devices of a vehicle |
| (exclusion constraint index) | gist (device_id, tstzrange) | Assignment of a device at an instant |

## 4.5 Table gps_position

| Item | Value |
|---|---|
| **Purpose** | Every accepted reading, after normalisation, with tenant and vehicle resolved at the reading's time. |
| **Created in** | V2 (columns total_distance_m, distance_source, ignition, odometer_m added in V5) |
| **Written by / read by** | gps-position-storage-service / position APIs, report summary job, vehicle-day sheet, previous-reading fallback |
| **Write pattern** | Insert only; a duplicate (imei, recorded_at) is skipped (checked first, and enforced by the unique key). |
| **Expected volume** | About 216,000 rows a day for 150 vehicles at one reading a minute; about 6.5 million a day in compressed simulator timing. |

***Table 17:*** *gps_position columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **imei** | varchar(50) | N | | Device (logical reference to device.imei). |
| 3 | **tenant_code** | varchar(40) | Y | | Tenant at the reading's time; null when the device was unassigned. |
| 4 | **vehicle_registration_no** | varchar(50) | Y | | Vehicle at the reading's time; null when unassigned. |
| 5 | **recorded_at** | timestamptz | N | | Time the device recorded the reading. |
| 6 | **latitude** | double precision | N | | WGS-84 degrees, −90 to 90. |
| 7 | **longitude** | double precision | N | | WGS-84 degrees, −180 to 180. |

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 8 | **speed_kmph** | integer | N | | Reported speed, 0 to 150. |
| 9 | **movement_status** | varchar(10) | N | | MOVING or STOPPED. |
| 10 | **distance_from_previous_m** | double precision | N | | Counted distance since the previous in-order reading (0 for first, jitter, jump and late readings). |
| 11 | **seconds_since_previous** | bigint | N | | Seconds since the previous in-order reading (0 for first and late readings). |
| 12 | **arrived_out_of_order** | boolean | N | | True for a late reading. |
| 13 | **processed_at** | timestamptz | N | | When the normaliser produced the position. |
| 14 | **stored_at** | timestamptz | N | now() | When the row was inserted. |
| 15 | **total_distance_m** | double precision | Y | | Running distance total of the device (V5). |
| 16 | **distance_source** | varchar(20) | Y | | FIRST_READING, GPS, DEVICE_ODOMETER, JITTER_FILTERED, JUMP_FILTERED or OUT_OF_ORDER (V5). |
| 17 | **ignition** | boolean | Y | | Device ignition input, when wired (V5). |
| 18 | **odometer_m** | bigint | Y | | Device odometer, when reported (V5). |

***Table 18:*** *gps_position constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| gps_position_pkey | PK | (id) |
| gps_position_one_reading_per_device_per_moment | UK | (imei, recorded_at): Kafka redelivery stores a reading once (BR-005). |
| gps_position_latitude_range | CK | latitude BETWEEN -90 AND 90 |
| gps_position_longitude_range | CK | longitude BETWEEN -180 AND 180 |
| gps_position_movement_status_values | CK | movement_status IN ('MOVING','STOPPED') |

***Table 19:*** *gps_position indexes*

| Index | Columns | Serves |
|---|---|---|
| gps_position_vehicle_time | (vehicle_registration_no, recorded_at DESC) | Latest position and history of a vehicle |
| gps_position_stored_at | (stored_at DESC) | Recently stored positions |
| gps_position_recorded_at | (recorded_at) | Report job: one day of positions for all vehicles (V8) |
| (unique key index) | (imei, recorded_at) | Duplicate check; previous reading of a device after restart |

## 4.6 Table vehicle_motion_state

| Item | Value |
|---|---|
| **Purpose** | Working memory of the trip detector: one row per device. |
| **Created in** | V3 |
| **Written by / read by** | gps-trip-service (TripDetectionService, VehicleMotionStateStore) / vehicle inspector |
| **Write pattern** | Upsert in the same transaction as the gps_trip and gps_stop rows it produces; guarded by last_recorded_at < the new reading's time. |

*Table 20: vehicle_motion_state columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **imei** | varchar(50) | N | | Device (PK). |
| 2 | **assignment_id** | uuid | Y | | device_assignment of the newest reading; a change closes the open trip. Null: not in a vehicle. |
| 3 | **tenant_code** | varchar(40) | Y | | Tenant of the newest reading. |
| 4 | **vehicle_registration_no** | varchar(50) | Y | | Vehicle of the newest reading. |
| 5 | **motion_streak** | boolean | N | | True between “started moving” and “stopped” (moving state). |
| 6 | **event_recorded_at** | timestamptz | N | | When the vehicle last started moving or stopped. |
| 7 | **event_latitude** | double precision | N | | Where that happened. |
| 8 | **event_longitude** | double precision | N | | Where that happened. |
| 9 | **last_recorded_at** | timestamptz | N | | Newest reading applied; older or equal readings are skipped. |
| 10 | **reading_window** | jsonb | N | | In-order readings of the last min-duration (a few dozen small objects), oldest first. |
| 11 | **open_trip** | jsonb | Y | | The trip in progress (same values as its OPEN row in gps_trip), or null. |
| 12 | **open_stop** | jsonb | Y | | The stop in progress (same values as its OPEN row in gps_stop), or null. |
| 13 | **updated_at** | timestamptz | N | now() | Last update. |

*Table 21: vehicle_motion_state constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| vehicle_motion_state_pkey | PK | (imei) |

## 4.7 Table gps_trip

| Item | Value |
|---|---|
| **Purpose** | A trip: from the moment a vehicle starts moving until it stops. |
| **Created in** | V4 |
| **Written by / read by** | gps-trip-service / GET /api/trips, report summary job, vehicle-day sheet |
| **Write pattern** | Upsert on (imei, start_recorded_at): an OPEN trip is extended with every reading and finally CLOSED. |

*Table 22: gps_trip columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **imei** | varchar(50) | N | | Device. |
| 3 | **tenant_code** | varchar(40) | Y | | Tenant from the assignment valid at the trip's readings. |
| 4 | **vehicle_registration_no** | varchar(50) | Y | | Vehicle from the same assignment. |
| 5 | **start_recorded_at** | timestamptz | N | | Start, back-dated to the first movement. |

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 6 | **end_recorded_at** | timestamptz | N | | End (moves with every reading while OPEN). |
| 7 | **start_latitude / start_longitude** | double precision | N | | Start position. |
| 8 | **end_latitude / end_longitude** | double precision | N | | End position. |
| 9 | **distance_m** | double precision | N | | Sum of distance_from_previous_m of in-order readings; never end − start odometer. |
| 10 | **duration_s** | bigint | N | | end − start in seconds. |
| 11 | **max_speed_kmph** | integer | N | | Highest reported speed. |
| 12 | **avg_speed_kmph** | double precision | N | | distance ÷ duration. |
| 13 | **reading_count** | integer | N | | In-order readings in the trip. |
| 14 | **late_reading_count** | integer | N | 0 | Late readings that fell into the trip (not in distance or duration). |
| 15 | **status** | varchar(10) | N | | OPEN or CLOSED. |
| 16 | **updated_at** | timestamptz | N | now() | Last update. |

***Table 23:*** *gps_trip constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| gps_trip_pkey | PK | (id) |
| gps_trip_one_trip_per_device_per_start | UK | (imei, start_recorded_at): a replayed reading updates the same trip. |
| gps_trip_status_values | CK | status IN ('OPEN','CLOSED') |
| gps_trip_ends_after_it_starts | CK | end_recorded_at >= start_recorded_at |
| gps_trip_distance_not_negative | CK | distance_m >= 0 |

***Table 24:*** *gps_trip indexes*

| Index | Columns | Serves |
|---|---|---|
| gps_trip_vehicle_time | (vehicle_registration_no, start_recorded_at DESC) | Trips of a vehicle in a period |

## 4.8 Table gps_stop

| Item | Value |
|---|---|
| **Purpose** | A stop: the vehicle stays within the minimum distance for at least the minimum duration. |
| **Created in** | V4 |
| **Written by / read by** | gps-trip-service / GET /api/stops, report summary job, vehicle-day sheet |
| **Write pattern** | Upsert on (imei, start_recorded_at). |

***Table 25:*** *gps_stop columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **imei** | varchar(50) | N | | Device. |
| 3 | **tenant_code** | varchar(40) | Y | | Tenant at the stop. |

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 4 | **vehicle_registration_no** | varchar(50) | Y | | Vehicle at the stop. |
| 5 | **start_recorded_at** | timestamptz | N | | Start of the stop. |
| 6 | **end_recorded_at** | timestamptz | N | | End (moves while OPEN). |
| 7 | **latitude / longitude** | double precision | N | | Stop position. |
| 8 | **duration_s** | bigint | N | | end − start in seconds. |
| 9 | **reading_count** | integer | N | | Readings during the stop. |
| 10 | **status** | varchar(10) | N | | OPEN or CLOSED. |
| 11 | **updated_at** | timestamptz | N | now() | Last update. |

***Table 26:*** *gps_stop constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| gps_stop_pkey | PK | (id) |
| gps_stop_one_stop_per_device_per_start | UK | (imei, start_recorded_at) |
| gps_stop_status_values | CK | status IN ('OPEN','CLOSED') |
| gps_stop_ends_after_it_starts | CK | end_recorded_at >= start_recorded_at |

***Table 27:*** *gps_stop indexes*

| Index | Columns | Serves |
|---|---|---|
| gps_stop_vehicle_time | (vehicle_registration_no, start_recorded_at DESC) | Stops of a vehicle in a period |

## 4.9 Table gps_trip_late_reading

| Item | Value |
|---|---|
| **Purpose** | Every late reading the trip detector skipped, once; also the work list of the future trip repair job. |
| **Created in** | V4 |
| **Written by / read by** | gps-trip-service / (future) repair job |

***Table 28:*** *gps_trip_late_reading columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **imei** | varchar(50) | N | | Device. |
| 2 | **recorded_at** | timestamptz | N | | Time of the late reading. |
| 3 | **trip_start_recorded_at** | timestamptz | Y | | Start of the trip it fell into (logical reference to gps_trip); null if it fell into a stop or no trip. |
| 4 | **noted_at** | timestamptz | N | now() | When it was noted. |

***Table 29:*** *gps_trip_late_reading constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| gps_trip_late_reading_pkey | PK | (imei, recorded_at): a Kafka redelivery is not counted again. |

## 4.10 Table vehicle_event

| Item | Value |
|---|---|
| **Purpose** | Events raised by the stream consumers (over-speed, sensor mismatch, stoppage, parking, idling, ignition, harsh driving). |
| **Created in** | V5 |
| **Written by / read by** | Speed-violation, vehicle-state, stoppage and ignition consumers / GET /api/events, report summary job, console Alerts |
| **Write pattern** | Upsert on event_key: written OPEN when an episode starts and again CLOSED when it ends. |

***Table 30:*** *vehicle_event columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **event_key** | varchar(120) | N | | Natural key of the episode (type, device, start). |
| 3 | **imei** | varchar(50) | N | | Device. |
| 4 | **tenant_code** | varchar(40) | Y | | Tenant. |
| 5 | **vehicle_registration_no** | varchar(50) | Y | | Vehicle. |
| 6 | **event_type** | varchar(40) | N | | OVERSPEED, SPEED_SENSOR_MISMATCH, STOPPAGE, PARKED, IDLING, IGNITION_ON, IGNITION_OFF, HARSH_ACCELERATION, HARSH_BRAKING. |
| 7 | **severity** | varchar(10) | N | | INFO, LOW, MEDIUM, HIGH, CRITICAL (not checked by the database). |
| 8 | **status** | varchar(10) | N | | OPEN or CLOSED. |
| 9 | **started_at** | timestamptz | N | | Episode start. |
| 10 | **ended_at** | timestamptz | Y | | Episode end; null while OPEN. |
| 11 | **latitude / longitude** | double precision | N | | Where the episode started. |
| 12 | **detected_by** | varchar(40) | N | | Consumer that raised it. |
| 13 | **details** | jsonb | N | '{}' | Type-specific facts (limit, peak speed, verdict, agents' findings …). |
| 14 | **created_at** | timestamptz | N | now() | First write. |
| 15 | **updated_at** | timestamptz | N | now() | Last write. |

***Table 31:*** *vehicle_event constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| vehicle_event_pkey | PK | (id) |
| vehicle_event_event_key_key | UK | (event_key): redelivery and the CLOSED update hit the same row. |
| vehicle_event_status_values | CK | status IN ('OPEN','CLOSED') |

***Table 32:*** *vehicle_event indexes*

| Index | Columns | Serves |
|---|---|---|
| vehicle_event_type_time | (event_type, started_at DESC) | Events of a type |
| vehicle_event_imei_time | (imei, started_at DESC) | Events of a vehicle |
| vehicle_event_recent | (updated_at DESC) | Newest first (GET /api/events) |

## 4.11 Table dead_letter

| Item | Value |
|---|---|
| **Purpose** | Permanent archive of every record a consumer or Kafka Streams application could not handle. |
| **Created in** | V6 |
| **Written by / read by** | gps-dead-letter-archive / Storage → Dead letters, GET /api/dead-letters |
| **Write pattern** | Insert once per dead letter; afterwards only status, replay_count, last_replayed_at and resolution_note change. Never deleted (except demo reset). |

***Table 33:*** *dead_letter columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **source** | varchar(20) | N | | Listener or Kafka Streams application that parked it. |
| 3 | **original_topic** | varchar(250) | Y | | Topic the record came from. |
| 4 | **original_partition** | integer | Y | | Its partition. |
| 5 | **original_offset** | bigint | Y | | Its offset. |
| 6 | **consumer_group** | varchar(250) | Y | | Group that failed. |
| 7 | **record_key** | varchar(250) | Y | | Record key (IMEI). |
| 8 | **record_value** | text | Y | | Record value exactly as received. |
| 9 | **exception_class** | varchar(500) | Y | | Error type. |
| 10 | **exception_message** | text | Y | | Error message. |
| 11 | **stacktrace** | text | Y | | Stack trace. |
| 12 | **parked_at** | timestamptz | N | | When it was parked. |
| 13 | **dead_letter_partition** | integer | N | | Partition in gps.dead-letters. |
| 14 | **dead_letter_offset** | bigint | N | | Offset in gps.dead-letters. |
| 15 | **status** | varchar(10) | N | 'PARKED' | PARKED, REPLAYED or DISCARDED. |
| 16 | **replay_count** | integer | N | 0 | Times replayed. |
| 17 | **last_replayed_at** | timestamptz | Y | | Last replay. |
| 18 | **resolution_note** | text | Y | | Why it was discarded (required for DISCARDED). |
| 19 | **archived_at** | timestamptz | N | now() | When the row was written. |

***Table 34:*** *dead_letter constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| dead_letter_pkey | PK | (id) |
| dead_letter_status_values | CK | status IN ('PARKED','REPLAYED','DISCARDED') |
| dead_letter_archived_once | UK | (dead_letter_partition, dead_letter_offset, parked_at): archived once, even if the topic is recreated. |

***Table 35:*** *dead_letter indexes*

| Index | Columns | Serves |
|---|---|---|
| dead_letter_status_time | (status, parked_at DESC) | Parked letters, newest first |

| Index | Columns | Serves |
|---|---|---|
| dead_letter_topic_time | (original_topic, parked_at DESC) | Replay the oldest parked of a topic |
| dead_letter_key | (record_key) | Dead letters of a device |

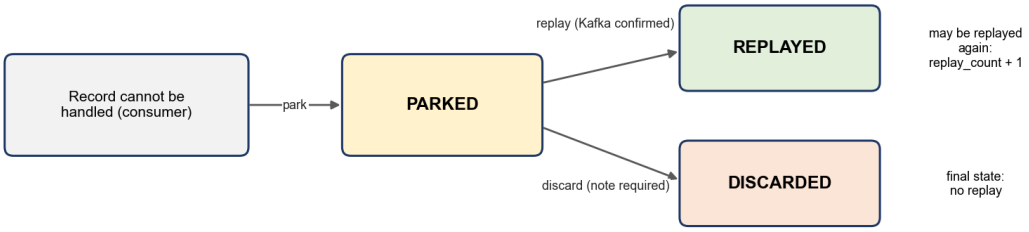



***Figure 3:*** *Status life cycle of dead_letter*

## 4.12 Table rejected_frame

| Item | Value |
|---|---|
| **Purpose** | Permanent archive of frames the normaliser refused, with the reason and the payload exactly as received. |
| **Created in** | V6 |
| **Written by / read by** | gps-rejected-frame-logger / Storage → Rejected frames, GET /api/rejected-frames |

***Table 36:*** *rejected_frame columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **imei** | varchar(50) | Y | | IMEI if it could be read. |
| 3 | **rejection_reason** | text | N | | Reason (for example “Latitude 95.0 is outside -90..90”). |
| 4 | **original_payload** | text | N | | Frame as received. |
| 5 | **rejected_at** | timestamptz | N | | Time of rejection. |
| 6 | **payload_hash** | char(32) | N | | Hash of the payload, for de-duplication. |
| 7 | **archived_at** | timestamptz | N | now() | When the row was written. |

***Table 37:*** *rejected_frame constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| rejected_frame_pkey | PK | (id) |
| rejected_frame_archived_once | UK | (rejected_at, payload_hash): a redelivered rejection is stored once. |

***Table 38:*** *rejected_frame indexes*

| Index | Columns | Serves |
|---|---|---|
| rejected_frame_time | (rejected_at DESC) | Newest rejections |
| rejected_frame_imei_time | (imei, rejected_at DESC) | Rejections of a device |

## 4.13 Table connection_setting

| Item | Value |
|---|---|
| **Purpose** | External addresses changed from the console while the service runs; a missing row means the application.yml default. |
| **Created in** | V7 |
| **Why in PostgreSQL** | The Redis address cannot be kept in the Redis it points to. |

*Table 39: connection_setting columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **name** | varchar(40) | N | | redis or valhalla. |
| 2 | **url** | varchar(500) | N | | redis://host:port/database or http://host:port. |
| 3 | **updated_at** | timestamptz | N | now() | Last change. |

*Table 40: connection_setting constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| connection_setting_pkey | PK | (name) |

## 4.14 Table vehicle_day_summary

| Item | Value |
|---|---|
| **Purpose** | One row per vehicle per day (and generation): the source of all day and month reports. |
| **Created in** | V8 |
| **Written by / read by** | DaySummaryJob (today and yesterday every 5 minutes; backfill at start; on demand) / GET /api/reports/* |
| **Generations** | 0 = live row, recomputed from the raw tables. Clear database freezes the live rows into a new generation > 0 before emptying the raw tables; a report adds all generations of a day. |
| **Expected size** | About 150 rows a day (4,650 a month) for 150 vehicles. |

*Table 41: vehicle_day_summary columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **day** | date | N | | Report day (Asia/Kolkata). |
| 2 | **imei** | varchar(50) | N | | Device. |
| 3 | **generation** | integer | N | 0 | 0 live; > 0 frozen by Clear database. |
| 4 | **tenant_code** | varchar(40) | Y | | Tenant. |
| 5 | **vehicle_registration_no** | varchar(50) | Y | | Vehicle. |
| 6 | **readings** | integer | N | | Positions in the day. |
| 7 | **first_seen_at / last_seen_at** | timestamptz | Y | | First and last reading of the day. |
| 8 | **distance_m** | double precision | N | | Distance driven. |
| 9 | **moving_s / stopped_s** | bigint | N | | Driving and standing time (long silences excluded). |
| 10 | **max_speed_kmph** | integer | N | | Highest speed. |

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 11 | **fast_distance_m** | double precision | N | | Distance driven above economical_speed_kmph. |
| 12 | **economical_speed_kmph** | integer | N | | Economical speed in force when computed. |
| 13 | **idle_s** | bigint | N | | Time in IDLING episodes. |
| 14 | **stops / stop_s** | integer / bigint | N | | Number and total time of stops. |
| 15 | **engine_on_stops** | integer | N | | Stops with the engine running. |
| 16 | **trips** | integer | N | | Trips started in the day. |
| 17 | **overspeed_events / harsh_events** | integer | N | | Event counts. |
| 18 | **computed_at** | timestamptz | N | now() | When computed. |

***Table 42:*** *vehicle_day_summary constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| vehicle_day_summary_pkey | PK | (day, imei, generation) |

***Table 43:*** *vehicle_day_summary indexes*

| Index | Columns | Serves |
|---|---|---|
| vehicle_day_summary_vehicle | (imei, day) | Month of one vehicle |

## 4.15 Table report_setting

| Item | Value |
|---|---|
| **Purpose** | Cost assumptions edited on Reports → Cost assumptions; a missing row means the built-in default. Not touched by Clear database. |
| **Created in** | V8 |

***Table 44:*** *report_setting columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **name** | varchar(60) | N | | Setting name (Table below). |
| 2 | **value** | varchar(100) | N | | Value as text. |
| 3 | **updated_at** | timestamptz | N | now() | Last change. |

***Table 45:*** *report_setting constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| report_setting_pkey | PK | (name) |

***Table 46:*** *report_setting names and defaults*

| name | Default | Validation |
|---|---|---|
| fuel-type | diesel | diesel or petrol |
| price-per-litre | 92.0 | ≥ 0 (rupees) |
| mileage-km-per-litre | 4.0 | > 0 |

| name | Default | Validation |
|---|---|---|
| idle-litres-per-hour | 2.5 | ≥ 0 |
| economical-speed-kmph | 60 | 10 to 150; a change rebuilds stored days |
| fast-driving-penalty-percent | 20.0 | ≥ 0 |
| co2-kg-per-litre | 2.68 (petrol 2.31) | ≥ 0 |

## 4.16 Tables added by V9 to V11

V9 adds transport management (three sign-in roles, places and shipments), V10 makes place the geofence master and adds the visits the geofence module finds, V11 adds the road route plans that the route-deviation and waypoints modules follow. tenant gains two columns in V9: kind (GPS_PROVIDER, TRANSPORTER, CONSIGNER; default TRANSPORTER) and external_code (the company's number in the TMS file; unique per kind when present).

## 4.17 Table app_user

| Item | Value |
|---|---|
| **Purpose** | Who may sign in, and as what: DEVELOPER (the GPS provider, sees everything), TRANSPORTER or CONSIGNER (one company each, sees only its data). |
| **Created in** | V9 |
| **Written by / read by** | PortalAdminController (users), demo accounts at start when a role has none / AuthController (sign-in) |

***Table 47:*** *app_user columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | uuid | N | gen_random_uuid() | Surrogate key. |
| 2 | **role** | varchar(20) | N | | DEVELOPER, TRANSPORTER or CONSIGNER. |
| 3 | **username** | varchar(80) | N | | User id typed at sign-in (unique per role, case-insensitive). |
| 4 | **display_name** | varchar(120) | N | | Name shown in the console. |
| 5 | **tenant_id** | uuid | Y | | FK tenant: the user's company (required except for DEVELOPER). |
| 6 | **password_hash** | varchar(200) | N | | PBKDF2-HMAC-SHA256 with salt; never the password. |
| 7 | **active** | boolean | N | true | False = may not sign in. |
| 8 | **created_at / last_login_at** | timestamptz | N / Y | now() / – | Created; last successful sign-in. |

***Table 48:*** *app_user constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| app_user_pkey | PK | (id) |
| app_user_role_values | CK | role IN ('DEVELOPER','TRANSPORTER','CONSIGNER') |
| app_user_company_for_business_roles | CK | role = 'DEVELOPER' OR tenant_id IS NOT NULL |
| app_user_role_username | UK (index) | (role, lower(username)) |

## 4.18 Table auth_key

| Item | Value |
|---|---|
| **Purpose** | The single secret that signs sign-in tokens (HMAC). Made by the migration, so it survives restarts and never sits in a configuration file; replacing it signs everyone out. |
| **Created in** | V9 |

*Table 49: auth_key columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | smallint | N | | Always 1. |
| 2 | **secret** | varchar(128) | N | random | Signing key. |
| 3 | **created_at** | timestamptz | N | now() | When made. |

*Table 50: auth_key constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| auth_key_pkey | PK | (id) |
| auth_key_single_row | CK | id = 1 |

## 4.19 Table place

| Item | Value |
|---|---|
| **Purpose** | The geofence master: plants, customers, yards, checkpoints and zones. A shipment file names its origin and destination; the name finds the place by code. Every active place is a geofence of the geofence module and can be a stop of a route plan. |
| **Created in** | V9 (circle, ORIGIN / DESTINATION); V10 adds shapes, roles, dwell and speed limits and the 87 smart_truck fences |
| **Written by / read by** | Live geofences page (PUT /api/geofences/{code}), import from smart_truck / GeofenceDirectory (index rebuilt within 30 s of a change), RoutePlanService |

*Table 51: place columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **code / name** | varchar(255) | N | | Normalised name the files use ("ADITYAPUR") / display name. |
| 3 | **role** | varchar(12) | N | | ORIGIN, DESTINATION, YARD, CHECKPOINT, ZONE. |
| 4 | **shape** | varchar(10) | N | 'CIRCLE' | CIRCLE or POLYGON (V10). |
| 5 | **polygon** | jsonb | Y | | [[latitude, longitude], …] around the border, at least 3 corners (V10). |
| 6 | **latitude / longitude** | double precision | N | | Centre (for a polygon: the average of its corners). |
| 7 | **radius_m** | integer | N | | Circle radius, 50 to 50,000 m; for a polygon the covering radius. |
| 8 | **dwell_alert_min** | integer | Y | | Inside longer raises GEOFENCE_DWELL; null = the role's default, 0 = never (V10). |
| 9 | **speed_limit_kmph** | integer | Y | | Limit inside the fence (5 to 150); the strictest applies to over-speed (V10). |

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 10 | **source / source_detail** | varchar(20) / varchar(30) | N / Y | 'MANUAL' | MANUAL or SMART_TRUCK; how smart_truck derived it (plant, anchor, gazetteer). |
| 11 | **external_ref** | varchar(60) | Y | | smart_truck:{id} for copied fences (V10). |
| 12 | **trips_seen / spread_m** | integer | Y | | smart_truck evidence: trips seen at the fence, spread of their pings (V10). |
| 13 | **active** | boolean | N | true | Inactive places are not geofences. |
| 14 | **created_at / updated_at** | timestamptz | N | now() | Created; last change (the directory's fingerprint). |

***Table 52:*** *place constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| place_pkey | PK | (id) |
| place_once_per_role | UK | (code, role) |
| place_role_values | CK | role IN ('ORIGIN','DESTINATION','YARD','CHECKPOINT','ZONE') |
| place_shape_values | CK | shape IN ('CIRCLE','POLYGON') |
| place_radius_sensible | CK | shape = 'POLYGON' OR radius_m BETWEEN 50 AND 50000 |
| place_polygon_present | CK | shape = 'CIRCLE' OR polygon is a JSON array of at least 3 corners |
| place_position_valid | CK | latitude BETWEEN -90 AND 90 AND longitude BETWEEN -180 AND 180 |
| place_dwell_sensible / place_speed_sensible | CK | dwell_alert_min ≥ 0; speed_limit_kmph BETWEEN 5 AND 150 |

## 4.20 Table shipment_import

| Item | Value |
|---|---|
| **Purpose** | One row per uploaded shipment file (Excel, CSV or the smart_truck tta_trips layout): what it changed and the problems found. |
| **Created in** | V9 |
| **Written by / read by** | ShipmentImportService / Transporter → Shipments → Upload, import history |

***Table 53:*** *shipment_import columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **file_name / file_sha256 / format** | varchar | N | | The file, its hash (an identical file is recognised), XLSX, CSV or SMART_TRUCK. |
| 3 | **uploaded_by / uploaded_role / tenant_id** | varchar / varchar / uuid | N / N / Y | | Who uploaded it, as what, for which company. |
| 4 | **rows_total, rows_created, rows_updated, rows_unchanged, rows_failed** | integer | N | | What the upload did. |
| 5 | **problems / column_mapping** | jsonb | N | '[]' / '{}' | Row problems; the column mapping used. |
| 6 | **uploaded_at** | timestamptz | N | now() | When. |

## 4.21 Table shipment

| Item | Value |
|---|---|
| **Purpose** | One trip of one truck carrying one consigner's goods, as the transporter's file describes it (the smart_truck tta_trips layout), plus what the GPS says about it. |
| **Created in** | V9 |
| **Identity** | (transporter_tenant_id, trip_no): the same row uploaded again updates its shipment, so a file uploaded twice cannot duplicate. |
| **Written by / read by** | ShipmentImportService; GPS columns by module shipment-tracking (always the earliest reading that proves them) / portals, RoutePlanService (a plan per open shipment) |

***Table 54:*** *shipment columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id / trip_no / source** | bigint / varchar(60) / varchar(12) | N | | Key; trip number in the file; where it came from. |
| 2 | **transporter_tenant_id / consigner_tenant_id** | uuid | N / Y | | FK tenant: the truck company and the goods owner. |
| 3 | **transporter_code, consigner_code, consigner_unit, consignee_name, consignee_address** | varchar | Y | | As written in the file. |
| 4 | **vehicle_registration_no / device_imei** | varchar(50) | N / Y | | The truck; the device named in the file (the mapped device wins). |
| 5 | **origin_name / destination_name / load_plant** | varchar(255) | Y | | Names from the file. |
| 6 | **origin_place_id / destination_place_id** | bigint | Y | | FK place: the geofences found for those names. |
| 7 | **booked_at / dispatched_at / eta_at** | timestamptz | Y / N / Y | | Booking, gate-out, the TMS's ETA. |
| 8 | **reported_ata_at / reported_end_at / source_status / close_reason** | timestamptz / varchar | Y | | What the TMS reported later. |
| 9 | **invoice_no, shipment_ref, gate_entry_no, card_id, route_id, material, quantity, driver_name, driver_mobile, vehicle_type, trip_class, trip_type** | varchar / numeric | Y | | Further columns of the file. |
| 10 | **left_origin_at / outside_destination_at / arrived_at** | timestamptz | Y | | GPS: cleared the origin fence; last outside the destination; entered it. |
| 11 | **last_reading_at, last_latitude, last_longitude, last_speed_kmph** | timestamptz / double / integer | Y | | GPS: the truck's latest reading during the trip. |
| 12 | **closed_at / closed_reason** | timestamptz / varchar(60) | Y | | Closed (arrived, a later trip of the truck, by hand). |
| 13 | **row_hash / import_id** | varchar(64) / bigint | N / Y | | Hash of the file row (unchanged rows are skipped); FK shipment_import. |
| 14 | **created_at / updated_at** | timestamptz | N | now() | Created; last change. |

*Table 55: shipment constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| shipment_pkey | PK | (id) |
| shipment_trip_once_per_transporter | UK | (transporter_tenant_id, trip_no) |

*Table 56: shipment indexes*

| Index | Columns | Serves |
|---|---|---|
| shipment_vehicle_dispatch / shipment_device_dispatch | (vehicle_registration_no \| device_imei, dispatched_at DESC) | Shipments of a reading's truck (module shipment-tracking) |
| shipment_consigner / shipment_transporter | (… _tenant_id, dispatched_at DESC) | Portal lists |

## 4.22 Table geofence_visit

| Item | Value |
|---|---|
| **Purpose** | Every visit to and drive-through of a geofence, as the geofence module finds it live: the source of the live board, the entry & exit log and the day-wise geofence report. |
| **Created in** | V10 |
| **Identity** | (imei, place_id, kind, entered_at): a visit found twice (a redelivery, a replay) is one row. |
| **Stamps** | entered_at = first reading inside, exited_at = last reading inside; the gaps to the readings outside are their uncertainty. |
| **No foreign key** | place_id is logical and code, name and role are copied, so a fence deleted later keeps its history. |

*Table 57: geofence_visit columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key. |
| 2 | **imei / vehicle_registration_no / tenant_code** | varchar | N / Y / Y | | The device and its truck and company at the time. |
| 3 | **place_id, place_code, place_name, place_role** | bigint / varchar | N | | The fence as it was. |
| 4 | **kind** | varchar(6) | N | | VISIT (inside 15 min: believed) or PASS (through, or over it between two readings). |
| 5 | **entered_at / exited_at** | timestamptz | N / Y | | Null exited_at = inside now. |
| 6 | **entry_gap_s / exit_gap_s** | bigint | Y | | Seconds to the reading outside: uncertainty of each stamp. |
| 7 | **entry_latitude, entry_longitude, exit_latitude, exit_longitude** | double precision | N / Y | | Where. |
| 8 | **dwell_alert_at / dwell_limit_min** | timestamptz / integer | Y | | When GEOFENCE_DWELL was raised, against which limit. |
| 9 | **inferred / exit_reason** | boolean / varchar(20) | N / Y | false | A pass worked out between two readings; HELD, ESCAPED, FENCE_REMOVED, REPLAY_CUT. |
| 10 | **created_at / updated_at** | timestamptz | N | now() | Created; last change. |

*Table 58: geofence_visit constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| geofence_visit_once | UK | (imei, place_id, kind, entered_at) |
| geofence_visit_kind_values | CK | kind IN ('VISIT','PASS') |
| geofence_visit_period_valid | CK | exited_at IS NULL OR exited_at >= entered_at |

*Table 59: geofence_visit indexes*

| Index | Columns | Serves |
|---|---|---|
| geofence_visit_open | (place_id) WHERE exited_at IS NULL | Who is inside which fence now |
| geofence_visit_time / _place_time / _vehicle_time / _imei_time | (… , entered_at DESC) | Day report; logs per fence, truck or device |

## 4.23 Table route_plan

| Item | Value |
|---|---|
| **Purpose** | One trip of one truck along a planned road route: the line from the routing engine (Valhalla on the HP; GraphHopper on this laptop as fallback), its per-instruction timings, and the live numbers the route-deviation module writes when they change. |
| **Created in** | V11 |
| **Made by** | RoutePlanService: every open shipment with an origin and a destination geofence (within 30 s), or by hand (Plan a route). A new plan closes the truck's other active plan. |
| **Versions** | version 1 is the dispatch plan; a ROUTE_ALTERNATE (another road about as good) replaces the line from where the truck was: version + 1, offset_m = distance already done; first_shape keeps the original. |
| **Read by** | RoutePlanDirectory (active plans in memory, reloaded within 30 s of a change), RouteLiveService (strips, trip map), portals |

*Table 60: route_plan columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id** | bigint identity | N | generated | Surrogate key; the episode scope of the plan's events. |
| 2 | **shipment_id** | bigint | Y | | FK shipment (ON DELETE CASCADE); null for a plan made by hand. One plan per shipment. |
| 3 | **imei / vehicle_registration_no / tenant_code** | varchar | Y / N / Y | | The truck followed. |
| 4 | **source / status** | varchar(12) | N | – / 'ACTIVE' | SHIPMENT or MANUAL; ACTIVE, ARRIVED, CANCELLED, CLOSED. |
| 5 | **origin_name, origin_place_id, origin_lat, origin_lon** | varchar / bigint / double | Y / Y / N / N | | Start (FK place when a geofence). |
| 6 | **destination_name, destination_place_id, destination_lat, destination_lon, destination_radius_m** | varchar / bigint / double | Y / Y / N / N / N | | Destination; arrival = a reading inside its geofence, or within the radius of a point. |
| 7 | **engine / costing** | varchar(20) | N | | valhalla or graphhopper; truck. |
| 8 | **version / reason** | integer / varchar(20) | N | 1 / – | DISPATCH, MANUAL or ALTERNATE. |

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 9 | **shape / timings** | text / jsonb | N | – / '[]' | Current line (encoded polyline, precision 6); [[from vertex, to vertex, seconds], …] of the engine's instructions. |
| 10 | **length_m / planned_s / offset_m** | double precision | N | – / – / 0 | Current line's length and planned driving time; distance done before this version. |
| 11 | **first_shape / first_length_m / first_planned_s** | text / double | N | | The dispatch plan, kept after re-plans. |
| 12 | **initial_pace** | double precision | N | | Actual / planned driving time to start with: the lane's median of 3+ arrived trips, else 1.0. |
| 13 | **planned_start_at** | timestamptz | N | | Dispatch time (shipment) or when planned. |
| 14 | **route_status** | varchar(12) | N | 'WAITING' | WAITING, ON_ROUTE, SUSPECT, OFF_ROUTE, ARRIVED. |
| 15 | **departed_at / promised_eta_at / eta_at / arrived_at** | timestamptz | Y | | Left the origin; ETA at departure; latest shown ETA; arrival. |
| 16 | **pace, chainage_m, cross_track_m** | double precision | Y | | Live pace; progress along the line; distance from it. |
| 17 | **last_reading_at, last_lat, last_lon** | timestamptz / double | Y | | Latest reading applied. |
| 18 | **deviations, deviation_km, deviation_minutes, wrong_ways, alternates** | integer / double | N | 0 | Trip totals. |
| 19 | **created_by / created_at / updated_at** | varchar / timestamptz | Y / N / N | – / now() | Who planned it; created; last change. |

***Table 61:*** *route_plan constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| route_plan_pkey | PK | (id) |
| route_plan_status_values / _source_values / _route_status_values | CK | The value sets of Section 6. |
| route_plan_positive | CK | length_m > 0 AND planned_s > 0 AND destination_radius_m > 0 |
| route_plan_one_per_shipment | UK (partial) | (shipment_id) WHERE shipment_id IS NOT NULL |

***Table 62:*** *route_plan indexes*

| Index | Columns | Serves |
|---|---|---|
| route_plan_active_vehicle / route_plan_active_imei | (vehicle_registration_no \| imei) WHERE status = 'ACTIVE' | A reading's plan (directory reload) |
| route_plan_recent | (planned_start_at DESC) | Live strips of the last hours |
| route_plan_lane | (origin_place_id, destination_place_id) WHERE status = 'ARRIVED' | The lane's pace history |

## 4.24 Table route_stop

| Item | Value |
|---|---|
| **Purpose** | The stops of a route plan, in the order the line meets them: waypoints planned by hand (MANUAL) and the geofences the planned line crosses (FENCE). |
| **Created in** | V11 |

| Item | Value |
|---|---|
| **Written by** | RoutePlanService (made with the plan; after a re-plan, FENCE stops not reached are found again) and module waypoints (status) |

*Table 63: route_stop columns*

| # | Column | Type | Null | Default | Description |
|---|---|---|---|---|---|
| 1 | **id / plan_id / seq** | bigint / bigint / integer | N | | Key; FK route_plan (ON DELETE CASCADE); order within the plan. |
| 2 | **kind / place_id / name** | varchar(10) / bigint / varchar(255) | N / Y / N | | MANUAL or FENCE; FK place when a geofence. |
| 3 | **latitude, longitude, radius_m** | double precision | N | | The stop's area when it is not a geofence. |
| 4 | **chainage_m** | double precision | N | | Distance along the current line; -1 when the line does not pass it. |
| 5 | **stay_min** | integer | N | 0 | Planned stay (counted in the ETA). |
| 6 | **status** | varchar(10) | N | 'PENDING' | PENDING, INSIDE, ARRIVED, DEPARTED, PASSED, MISSED. |
| 7 | **arrived_at / departed_at / missed_at / minutes_inside / out_of_order** | timestamptz / integer / boolean | Y (out_of_order N) | out_of_order false | What the truck did there. |
| 8 | **updated_at** | timestamptz | N | now() | Last change. |

*Table 64: route_stop constraints*

| Constraint | Kind | Definition and purpose |
|---|---|---|
| route_stop_once | UK | (plan_id, seq) |
| route_stop_kind_values / route_stop_status_values | CK | The value sets of Section 6. |

# 5 Relationships and Referential Integrity

***Table 65:*** *Relationships*

| Parent | Child | Cardinality | Enforced by | Rule |
|---|---|---|---|---|
| tenant | vehicle | 0..1 : N | FK | A vehicle may have no owner (market vehicle). |
| device | device_assignment | 1 : N | FK + EX | Periods of one device never overlap. |
| vehicle | device_assignment | 1 : N | FK | A vehicle may carry several devices (GPS, fuel …). |
| device (imei) | gps_position, vehicle_motion_state, gps_trip, gps_stop, gps_trip_late_reading, vehicle_event, vehicle_day_summary, rejected_frame | 1 : N | Logical | Rows are kept even if the device is retired. |
| tenant (code) | telemetry tables (tenant_code) | 1 : N | Logical | Code is never reused, so history stays unambiguous. |
| vehicle (registration_no) | telemetry tables (vehicle_registration_no) | 1 : N | Logical | Copied at the reading's time (BR-004). |
| device_assignment | vehicle_motion_state.assignment_id | 1 : N | Logical | A change of assignment closes the open trip. |
| gps_trip | gps_trip_late_reading.trip_start_recorded_at | 1 : N | Logical (imei + start) | Late readings of a trip. |
| tenant | app_user, shipment (transporter, consigner) | 1 : N | FK | A business user belongs to one company. |
| place | shipment (origin, destination), route_plan (origin, destination), route_stop | 1 : N | FK | A place in use is deactivated, not deleted. |
| place | geofence_visit | 1 : N | Logical (place_id + copied name) | History outlives the fence. |
| shipment_import | shipment | 1 : N | FK | The upload that last wrote the row. |
| shipment | route_plan | 1 : 0..1 | FK + partial UK, cascade | One plan per shipment. |
| route_plan | route_stop | 1 : N | FK, cascade | Stops are deleted with their plan. |

Logical references are deliberate (SRS DR-002). The application guarantees them as follows: tenant and vehicle are resolved from device_assignment for every reading (or left null with a WARN log); codes and registration numbers are never reused; master-data rows are never deleted while telemetry refers to them.

# 6 Value Sets

*Table 66: Enumerated values*

| Column(s) | Allowed values | Checked by |
|---|---|---|
| tenant.status | ACTIVE, SUSPENDED, CLOSED | CK |
| device.kind | GPS, FUEL, TEMPERATURE, DOOR, OBD | CK |
| gps_position.movement_status | MOVING, STOPPED | CK |
| gps_position.distance_source | FIRST_READING, GPS, DEVICE_ODOMETER, JITTER_FILTERED, JUMP_FILTERED, OUT_OF_ORDER | Application (enum) |
| gps_trip.status, gps_stop.status, vehicle_event.status | OPEN, CLOSED | CK |
| vehicle_event.event_type | OVERSPEED, SPEED_SENSOR_MISMATCH, STOPPAGE, PARKED, IDLING, IGNITION_ON, IGNITION_OFF, HARSH_ACCELERATION, HARSH_BRAKING, SHIPMENT_DEPARTED, SHIPMENT_ARRIVED, SHIPMENT_DELAYED, GEOFENCE_ENTER, GEOFENCE_EXIT, GEOFENCE_PASS, GEOFENCE_DWELL, ROUTE_DEVIATION, ROUTE_ALTERNATE, WRONG_DIRECTION, ETA_CHANGED, DESTINATION_REACHED, WAYPOINT_ARRIVED, WAYPOINT_DEPARTED, WAYPOINT_MISSED | Application (enum) |
| vehicle_event.severity | INFO, LOW, MEDIUM, HIGH, CRITICAL | Application (enum) |
| dead_letter.status | PARKED, REPLAYED, DISCARDED | CK |
| connection_setting.name | redis, valhalla | Application |
| tenant.kind | GPS_PROVIDER, TRANSPORTER, CONSIGNER | CK |
| app_user.role | DEVELOPER, TRANSPORTER, CONSIGNER | CK |
| place.role / place.shape | ORIGIN, DESTINATION, YARD, CHECKPOINT, ZONE / CIRCLE, POLYGON | CK |
| geofence_visit.kind | VISIT, PASS | CK |
| route_plan.status | ACTIVE, ARRIVED, CANCELLED, CLOSED | CK |
| route_plan.route_status | WAITING, ON_ROUTE, SUSPECT, OFF_ROUTE, ARRIVED | CK |
| route_plan.source / reason | SHIPMENT, MANUAL / DISPATCH, MANUAL, ALTERNATE | CK / Application |
| route_stop.kind / status | MANUAL, FENCE / PENDING, INSIDE, ARRIVED, DEPARTED, PASSED, MISSED | CK |

**Note:** event_type, severity and distance_source are checked only by the application. Adding CHECK constraints for them is recommended in a future migration (open issue DB-OI-02).

# 7 Index Strategy and Main Queries

Every frequent query is served by an index whose leading columns match its filter. The main access paths are:

***Table 67:*** *Main queries and the index that serves them*

| Query | Index | Frequency |
|---|---|---|
| Duplicate check before insert: exists (imei, recorded_at) | unique (imei, recorded_at) | Every position |
| Previous reading after restart: newest row of an imei | unique (imei, recorded_at) | First reading per device after start |
| Latest position of a vehicle | gps_position_vehicle_time | API, on demand |
| One day of positions of all vehicles (summary job) | gps_position_recorded_at | Every 5 minutes, two days |
| Trips / stops of a vehicle overlapping a period | gps_trip_vehicle_time, gps_stop_vehicle_time | API, reports |
| Newest events, events of a type or a vehicle | vehicle_event_recent, _type_time, _imei_time | Console polling (2 s) |
| Parked dead letters; oldest parked of a topic | dead_letter_status_time, dead_letter_topic_time | Console, replay |
| Month of one vehicle | vehicle_day_summary_vehicle | Reports |
| Shipments of a reading's truck | shipment_vehicle_dispatch, shipment_device_dispatch | Every reading (shipment-tracking) |
| Who is inside a fence now; visits overlapping a day | geofence_visit_open; geofence_visit_*_time | Live board 5 s; day report |
| Active plans (directory reload) | route_plan_active_vehicle, route_plan_active_imei | Within 30 s of a change |
| Plans of the last hours; the lane's pace | route_plan_recent; route_plan_lane | Live strips 5 s; each new plan |

```
-- Assignment valid at a reading's time (served by the exclusion-constraint gist index)
SELECT a.id, v.registration_no, t.code
  FROM device_assignment a
  JOIN device  d ON d.id = a.device_id
  JOIN vehicle v ON v.id = a.vehicle_id
  LEFT JOIN tenant t ON t.id = v.owner_tenant_id
 WHERE d.imei = :imei
   AND a.valid_from <= :recordedAt AND (a.valid_to IS NULL OR a.valid_to > :recordedAt);
```

# 8 Data Integrity, Idempotency and Concurrency

***Table 68:*** *Idempotency key of each consumer-written table*

| Table | Key | On redelivery |
|---|---|---|
| gps_position | (imei, recorded_at) | Existence check first; unique key as the last line of defence |
| gps_trip, gps_stop | (imei, start_recorded_at) | Upsert updates the same row |
| geofence_visit | (imei, place_id, kind, entered_at) | Upsert; a replay first cuts the visits after its start |
| route_plan | (shipment_id) partial unique | A shipment is planned once; live columns are overwritten |
| route_stop | (plan_id, seq) | Status update of the same row |
| vehicle_motion_state | imei + guard last_recorded_at < :recordedAt | 0 rows updated → transaction rolled back |
| gps_trip_late_reading | (imei, recorded_at) | Insert … ON CONFLICT DO NOTHING; late_reading_count not raised twice |
| vehicle_event | event_key | Upsert (OPEN then CLOSED) |
| dead_letter | (dead_letter_partition, dead_letter_offset, parked_at) | Stored once |
| rejected_frame | (rejected_at, payload_hash) | Stored once |
| vehicle_day_summary | (day, imei, generation) | Generation 0 replaced on every rebuild |

Guarded update used by the trip detector (one transaction per reading, together with the trip and stop upserts):

```
UPDATE vehicle_motion_state
   SET motion_streak = :motionStreak, event_recorded_at = :eventAt, event_latitude = :eventLat,
       event_longitude = :eventLon, last_recorded_at = :recordedAt, reading_window = :window::jsonb,
       open_trip = :openTrip::jsonb, open_stop = :openStop::jsonb, assignment_id = :assignmentId,
       tenant_code = :tenant, vehicle_registration_no = :vehicle, updated_at = now()
 WHERE imei = :imei
   AND last_recorded_at < :recordedAt;   -- 0 rows: a later or equal reading was already applied
```

Concurrency: all readings of one device are in one Kafka partition and therefore handled by one thread at a time, so rows keyed by IMEI have a single writer. Configuration rows (report_setting, connection_setting) are last-writer-wins.

# 9 Data Lifecycle: Retention, Clearing, Backup and Recovery

## 9.1 Retention

*Table 69: Retention by data store*

| Data | Retention | Basis |
|---|---|---|
| Kafka topics | 31 days, by age only | gps.kafka.retention; SRS NFR-CAP-002 |
| Telemetry and trip tables | Until deleted (no automatic deletion yet) | Retention policy is open: SRS NFR-PRV-003, OI-04 |
| dead_letter, rejected_frame | Indefinite | Never deleted except by demo reset (SRS FR-DLQ-008) |
| vehicle_day_summary | Indefinite | Small; frozen generations survive Clear database |
| Master data, settings | Indefinite | Survive Clear database and demo reset |
| Redis live state | Until replaced; history about 1000 entries per vehicle | Rebuilt from the next readings |

## 9.2 Clear Database and Demo Reset

*Table 70: What is emptied*

| Table | Clear database (keep reports) | Demo reset / clear without reports |
|---|---|---|
| gps_position, gps_trip, gps_stop, gps_trip_late_reading, vehicle_motion_state, vehicle_event | Emptied (TRUNCATE … RESTART IDENTITY) | Emptied |
| dead_letter, rejected_frame | Emptied | Emptied |
| vehicle_day_summary | Kept; live rows frozen into a new generation first | Emptied |
| tenant, vehicle, device, device_assignment | Kept | Kept |
| report_setting, connection_setting | Kept | Kept |

## 9.3 Backup and Recovery

- Proposed (SRS DR-008): a daily logical backup (pg_dump -Fc gps_tracking), kept 31 days, with a quarterly restore test.
- Recovery to a point in time within 31 days: restore the newest backup older than the target, then reset the consumer groups' offsets to the backup time and let them re-consume Kafka. Idempotency keys (Section 8) make the overlap harmless.
- Redis needs no backup: live state is rebuilt from the next readings; speed rules (gps:config:*) should be exported with the database backup.

## 9.4 Capacity Estimate

*Table 71: Growth estimate for 150 vehicles*

| Table | Rows per day | Approx. size per day (with indexes) |
|---|---|---|
| gps_position (1 reading/min) | 216,000 | about 60 to 80 MB |

| Table | Rows per day | Approx. size per day (with indexes) |
|---|---|---|
| gps_position (compressed simulator) | about 6,500,000 | about 2 GB; demo use only |
| gps_trip + gps_stop | a few thousand | under 2 MB |
| vehicle_event | a few hundred to thousands | under 2 MB |
| vehicle_day_summary | 150 | negligible |

**Note:** Sizes are estimates (about 250 to 350 bytes per gps_position row including indexes) and shall be confirmed with pg_total_relation_size after a week of normal operation.

# 10 Migration History and Change Rules

***Table 72:*** *Flyway migrations*

| Version | File | Change |
|---|---|---|
| V1 | V1__create_master_data_tables.sql | btree_gist; tenant, vehicle, device, device_assignment |
| V2 | V2__create_gps_position_table.sql | gps_position with checks, unique key and indexes |
| V3 | V3__create_vehicle_motion_state_table.sql | Trip-detector state |
| V4 | V4__create_gps_trip_and_gps_stop_tables.sql | gps_trip, gps_stop, gps_trip_late_reading |
| V5 | V5__distance_details_and_vehicle_events.sql | gps_position distance columns, ignition, odometer; vehicle_event |
| V6 | V6__dead_letter_and_rejected_frame_archive.sql | dead_letter, rejected_frame |
| V7 | V7__connection_setting.sql | connection_setting |
| V8 | V8__vehicle_day_summary_and_report_setting.sql | vehicle_day_summary, report_setting, index gps_position_recorded_at |

- An applied migration shall never be edited: Flyway's checksum check stops the service. Every change is a new file V9__<description>.sql, V10__ …
- Every new column or table shall be added to the JPA entities (where one exists) in the same change, because Hibernate validates the schema at start.
- A migration that rewrites a large table (for example adding a NOT NULL column with a default to gps_position) shall be reviewed for lock time and run outside peak hours.
- This document shall be updated in the same change as the migration (SRS Rules and Regulations, change control).

# 11 Non-Relational Data Stores

## 11.1 Redis Key Space

***Table 73:*** *Redis keys*

| Key pattern | Type | Written by | Content / lifetime |
|---|---|---|---|
| gps:live:vehicle:{imei} | hash | live-location | vehicle, tenant, lat, lon, speed, status, recordedAt, updatedAt; newer wins |
| gps:live:positions | geo set | live-location | One member per IMEI, for GEOSEARCH by radius |
| gps:state:vehicle:{imei} | hash | vehicle-state | Heading, acceleration, Kalman track, counters |
| gps:state:speed:{imei} | hash | speed-violation | Current episode, GPS window, verdict |
| gps:state:ignition:{imei} | hash | ignition | Ignition state, engine hours, idling |
| gps:state:stoppage:{imei} | hash | stoppage | Stay-point candidate or current stop |
| gps:history:{imei} | stream | vehicle-state | XADD per reading, XTRIM MAXLEN ~1000 |
| gps:events:{imei} | list | event recorder | Recent events of the vehicle |
| gps:context:weather:{cell} | string | stoppage agents | Weather of a 0.1° cell, 10-minute expiry |
| gps:config:speed-rules | set | speed rules API | Scopes that have a rule |
| gps:config:speed-rule:{scope} | hash | speed rules API | Rule fields (Section 3.9 of the SRS); kept by resets |
| gps:engine:stage:{stage} | hash | metrics mirror | Totals, rate, failures, last message; every 5 s |
| gps:engine:stage:{stage}:by-imei | hash | metrics mirror | Messages per IMEI for the stage |

Telemetry patterns deleted by the demo reset: gps:live:*, gps:engine:*, gps:state:*, gps:history:*, gps:events:*, gps:context:*. Configuration (gps:config:*) and other programs' keys are never deleted.

## 11.2 Kafka Topics

***Table 74:*** *Topics (all keyed by IMEI, 3 partitions, 31-day retention, zstd)*

| Topic | Value | Notes |
|---|---|---|
| gps.raw-device-frames | Raw frame JSON | Written through the ingest journal |
| gps.processed-positions | Processed position JSON | Transactional (exactly-once); timestamp = recordedAt |
| gps.rejected-frames | Rejected frame JSON | Transactional |
| gps.vehicle-events | Vehicle event JSON | Written through the ingest journal |
| gps.vehicle-activity | Activity window JSON | One final record per closed 5-minute window |
| gps.dead-letters | Original bytes + error headers | Archived to dead_letter |
| gps.shadow.processed-positions / rejected-frames | As above | Shadow mode only |

## 11.3 Kafka Streams State Stores

***Table 75:*** *State stores (RocksDB, each backed by a compacted changelog topic)*

| Store | Application | Content |
|---|---|---|
| previous-reading-store | normaliser | Newest reading per IMEI (distance and order) |
| seen-reading-store | normaliser | Window store of readings seen in the last 24 h (duplicates) |
| vehicle-table-store | analytics | KTable: latest snapshot per vehicle |
| activity-window-store | analytics | 5-minute windows, kept 6 h for queries |

# 12 External Source: smart_truck (Read-Only)

The simulator reads the legacy MySQL database smart_truck through a separate read-only pool, with SELECT statements only (SRS CON-002). Times in smart_truck are India time and are converted to UTC instants.

***Table 76:*** *smart_truck columns read*

| Table.column | Meaning | Becomes |
|---|---|---|
| tta_trips.i_trip_no | Trip number | Selects the latest trip per device |
| tta_trips.i_gps_ping_count | Pings in the trip | Filter: at least 200 (minimum-pings-per-trip) |
| tta_trip_gps.s_device_id | Device identifier | imei |
| tta_trip_gps.s_asset_id | Vehicle identifier | vehicle.registration_no |
| tta_trip_gps.s_entity_name | Transporter name | tenant.legal_name → tenant.code |
| tta_trip_gps.dt_message | Ping time (IST) | recordedAt (UTC) |
| tta_trip_gps.d_lat / d_long | Position | latitude / longitude |
| tta_trip_gps.i_speed | Speed km/h | speedKmph |
| tta_trip_gps.i_dist | Distance increment | Simulated odometer (for 50 % of devices) |

# 13 Open Issues

***Table 77:*** *Database open issues*

| ID | Issue | Owner |
|---|---|---|
| DB-OI-01 | Define and implement retention and deletion for telemetry tables (time-based partitions on gps_position are the likely solution). | Technical lead |
| DB-OI-02 | Add CHECK constraints for vehicle_event.event_type and severity and gps_position.distance_source. | Developer |
| DB-OI-03 | Decide the time-series store (ClickHouse is planned) and the hand-over of gps_position history. | Architect |
| DB-OI-04 | Master-data administration API: who may create, change and close tenants, vehicles, devices and assignments. | Product owner |
| DB-OI-05 | Automated backup and restore test (SRS DR-008). | Operations |

# 14 Appendices

## 14.1 Appendix A: Complete DDL

The exact text of the Flyway migrations, in order. This appendix is regenerated from the files whenever the document is built.

### 14.1.1 V1__create_master_data_tables.sql

```
-- Master data needed to turn a device IMEI into a vehicle and a tenant.
-- This is the first slice of the GPS master-data design: tenant, vehicle, device, device_assignment.
-- The remaining tables (tenant_role, branch, app_user, trip, shipment, tracking_grant ...) follow in
later migrations.

-- Needed for the "one vehicle per device at a time" exclusion constraint below.
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE tenant (
    id          uuid         PRIMARY KEY,
    code        varchar(40)  NOT NULL UNIQUE,
    legal_name  varchar(255) NOT NULL,
    status      varchar(20)  NOT NULL DEFAULT 'ACTIVE',
    created_at  timestamptz  NOT NULL DEFAULT now(),
    -- code is stamped on every GPS position, so it is short, lower-case and never re-used
    CONSTRAINT tenant_code_format CHECK (code ~ '^[a-z0-9][a-z0-9_-]*$'),
    CONSTRAINT tenant_status_values CHECK (status IN ('ACTIVE', 'SUSPENDED', 'CLOSED'))
);

CREATE TABLE vehicle (
    id              uuid        PRIMARY KEY,
    -- one truck is one truck: the plate is unique across the whole platform
    registration_no  varchar(50) NOT NULL UNIQUE,
    -- null = market vehicle (not owned by a tenant on the platform)
    owner_tenant_id  uuid        REFERENCES tenant (id),
    created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE device (
    id          uuid        PRIMARY KEY,
    imei        varchar(50) NOT NULL UNIQUE,
    kind        varchar(20) NOT NULL DEFAULT 'GPS',
    protocol    varchar(50),
    created_at  timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT device_kind_values CHECK (kind IN ('GPS', 'FUEL', 'TEMPERATURE', 'DOOR', 'OBD'))
);

-- Stored as periods, never as "current vehicle": a reading delivered late must be matched
-- to the vehicle the device was in at the time of the reading.
CREATE TABLE device_assignment (
    id          uuid        PRIMARY KEY,
    device_id   uuid        NOT NULL REFERENCES device (id),
    vehicle_id  uuid        NOT NULL REFERENCES vehicle (id),
    valid_from  timestamptz NOT NULL,
    valid_to    timestamptz,
    CONSTRAINT device_assignment_period_is_valid CHECK (valid_to IS NULL OR valid_to > valid_from),
    -- A device can be in only one vehicle at a time, so every reading resolves to exactly one truck.
    CONSTRAINT device_assignment_one_vehicle_per_device_at_a_time
        EXCLUDE USING gist (device_id WITH =, tstzrange(valid_from, valid_to) WITH &&)
);

CREATE INDEX device_assignment_vehicle ON device_assignment (vehicle_id);
```

### 14.1.2 V2__create_gps_position_table.sql

```
-- GPS positions after Flink processing, with the tenant and vehicle resolved from master data.
-- Tenant and vehicle are stored as readable codes (not foreign keys): telemetry is written fast,
-- read in reports, and will later move to a time-series store.

CREATE TABLE gps_position (
    id                      bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei                    varchar(50)      NOT NULL,
    tenant_code             varchar(40),
    vehicle_registration_no  varchar(50),
```

```
    recorded_at               timestamptz      NOT NULL,
    latitude                  double precision NOT NULL,
    longitude                 double precision NOT NULL,
    speed_kmph                integer          NOT NULL,
    movement_status           varchar(10)      NOT NULL,
    distance_from_previous_m  double precision NOT NULL,
    seconds_since_previous    bigint           NOT NULL,
    arrived_out_of_order      boolean          NOT NULL,
    processed_at              timestamptz      NOT NULL,
    stored_at                 timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT gps_position_latitude_range  CHECK (latitude BETWEEN -90 AND 90),
    CONSTRAINT gps_position_longitude_range CHECK (longitude BETWEEN -180 AND 180),
    CONSTRAINT gps_position_movement_status_values CHECK (movement_status IN ('MOVING', 'STOPPED')),
    -- Kafka delivers at-least-once, so the same reading can arrive twice; it is stored once.
    CONSTRAINT gps_position_one_reading_per_device_per_moment UNIQUE (imei, recorded_at)
);

CREATE INDEX gps_position_vehicle_time ON gps_position (vehicle_registration_no, recorded_at DESC);
CREATE INDEX gps_position_stored_at    ON gps_position (stored_at DESC);
```

### 14.1.3 V3__create_vehicle_motion_state_table.sql

```
-- State of the trip detection (consumer group gps-trip-service): one row per device.
-- Traccar keeps the same on tc_devices (motionstreak, motiontime, motionlatitude, motionlongitude)
-- plus the recent positions in memory. Here it is one row, updated in the SAME transaction as the
-- gps_trip / gps_stop rows it produces, so a trip can never be written twice or lost half-way.
--
-- Every update is guarded by "last_recorded_at < the new reading's time": a reading that is late,
-- or delivered again by Kafka, updates 0 rows and changes nothing.

CREATE TABLE vehicle_motion_state (
    imei                      varchar(50)      PRIMARY KEY,
    -- the device_assignment of the newest reading; a change closes the open trip (null: not in a
vehicle)
    assignment_id             uuid,
    tenant_code               varchar(40),
    vehicle_registration_no   varchar(50),
    -- true between a "started moving" and a "stopped" decision (Traccar's motionStreak)
    motion_streak             boolean          NOT NULL,
    -- where and when the vehicle last started moving or stopped
    event_recorded_at         timestamptz      NOT NULL,
    event_latitude            double precision NOT NULL,
    event_longitude           double precision NOT NULL,
    -- the newest reading applied: older or equal readings are skipped
    last_recorded_at          timestamptz      NOT NULL,
    -- in-order readings of the last minDuration (a few dozen small objects), oldest first.
    -- Named reading_window because WINDOW is a reserved word in PostgreSQL.
    reading_window            jsonb            NOT NULL,
    -- the trip or stop going on now (the same values as its OPEN row in gps_trip / gps_stop), or null
    open_trip                 jsonb,
    open_stop                 jsonb,
    updated_at                timestamptz      NOT NULL DEFAULT now()
);
```

### 14.1.4 V4__create_gps_trip_and_gps_stop_tables.sql

```
-- Trips and stops detected by consumer group gps-trip-service (Traccar computes these on request;
-- here they are written as the readings arrive). Tenant and vehicle are readable codes, as in
gps_position,
-- taken from the device_assignment valid at the time of the readings.
--
-- A trip or stop is identified by (imei, start_recorded_at): a replayed reading produces the same
key,
-- and the row is updated instead of stored twice.

CREATE TABLE gps_trip (
    id                        bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei                      varchar(50)      NOT NULL,
    tenant_code               varchar(40),
    vehicle_registration_no   varchar(50),
    start_recorded_at         timestamptz      NOT NULL,
    end_recorded_at           timestamptz      NOT NULL,
    start_latitude            double precision NOT NULL,
    start_longitude           double precision NOT NULL,
    end_latitude              double precision NOT NULL,
    end_longitude             double precision NOT NULL,
```

```
    -- sum of distance_from_previous of the in-order readings, never "end odometer - start odometer"
    distance_m                double precision NOT NULL,
    duration_s                bigint           NOT NULL,
    max_speed_kmph            integer          NOT NULL,
    avg_speed_kmph            double precision NOT NULL,
    reading_count             integer          NOT NULL,
    -- readings that arrived late for this trip; they are not in distance/duration (see
gps_trip_late_reading)
    late_reading_count        integer          NOT NULL DEFAULT 0,
    -- OPEN: the vehicle is still driving, the end moves with every reading. CLOSED: final.
    status                    varchar(10)      NOT NULL,
    updated_at                timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT gps_trip_status_values CHECK (status IN ('OPEN', 'CLOSED')),
    CONSTRAINT gps_trip_ends_after_it_starts CHECK (end_recorded_at >= start_recorded_at),
    -- Traccar's trips report once said -1020 m because of one late reading; this table refuses that.
    CONSTRAINT gps_trip_distance_not_negative CHECK (distance_m >= 0),
    CONSTRAINT gps_trip_one_trip_per_device_per_start UNIQUE (imei, start_recorded_at)
);

CREATE INDEX gps_trip_vehicle_time ON gps_trip (vehicle_registration_no, start_recorded_at DESC);

CREATE TABLE gps_stop (
    id                        bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei                      varchar(50)      NOT NULL,
    tenant_code               varchar(40),
    vehicle_registration_no   varchar(50),
    start_recorded_at         timestamptz      NOT NULL,
    end_recorded_at           timestamptz      NOT NULL,
    latitude                  double precision NOT NULL,
    longitude                 double precision NOT NULL,
    duration_s                bigint           NOT NULL,
    reading_count             integer          NOT NULL,
    status                    varchar(10)      NOT NULL,
    updated_at                timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT gps_stop_status_values CHECK (status IN ('OPEN', 'CLOSED')),
    CONSTRAINT gps_stop_ends_after_it_starts CHECK (end_recorded_at >= start_recorded_at),
    CONSTRAINT gps_stop_one_stop_per_device_per_start UNIQUE (imei, start_recorded_at)
);

CREATE INDEX gps_stop_vehicle_time ON gps_stop (vehicle_registration_no, start_recorded_at DESC);

-- Every late reading the trip detection skipped, once (a Kafka redelivery is not counted again).
-- trip_start_recorded_at points to the trip it fell into (null: it fell into a stop or no trip).
-- This is also the work list for a later "repair" job that recomputes those trips from gps_position.
CREATE TABLE gps_trip_late_reading (
    imei                      varchar(50)  NOT NULL,
    recorded_at               timestamptz  NOT NULL,
    trip_start_recorded_at    timestamptz,
    noted_at                  timestamptz  NOT NULL DEFAULT now(),
    PRIMARY KEY (imei, recorded_at)
);
```

### 14.1.5 V5__distance_details_and_vehicle_events.sql

```
-- Distance as Traccar's DistanceHandler computes it (see CalculateMovementStep), and what the device
reported.
ALTER TABLE gps_position
    ADD COLUMN total_distance_m double precision,
    ADD COLUMN distance_source  varchar(20),
    ADD COLUMN ignition         boolean,
    ADD COLUMN odometer_m       bigint;

-- Events raised by the stream consumers (over-speed, stoppage, parking, idling, ignition, harsh
driving ...).
-- An episode is written when it starts (OPEN) and again when it ends (CLOSED) with the same
event_key,
-- so the row is upserted - which also makes a Kafka redelivery harmless.
CREATE TABLE vehicle_event (
    id                        bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    event_key                 varchar(120) NOT NULL UNIQUE,
    imei                      varchar(50)  NOT NULL,
    tenant_code               varchar(40),
    vehicle_registration_no   varchar(50),
    event_type                varchar(40)  NOT NULL,
    severity                  varchar(10)  NOT NULL,
    status                    varchar(10)  NOT NULL,
    started_at                timestamptz  NOT NULL,
```

```
    ended_at                timestamptz,
    latitude                double precision NOT NULL,
    longitude               double precision NOT NULL,
    detected_by             varchar(40)  NOT NULL,
    details                 jsonb        NOT NULL DEFAULT '{}'::jsonb,
    created_at              timestamptz  NOT NULL DEFAULT now(),
    updated_at              timestamptz  NOT NULL DEFAULT now(),
    CONSTRAINT vehicle_event_status_values CHECK (status IN ('OPEN', 'CLOSED'))
);

CREATE INDEX vehicle_event_type_time   ON vehicle_event (event_type, started_at DESC);
CREATE INDEX vehicle_event_imei_time   ON vehicle_event (imei, started_at DESC);
CREATE INDEX vehicle_event_recent      ON vehicle_event (updated_at DESC);
```

### 14.1.6 V6__dead_letter_and_rejected_frame_archive.sql

```
-- Nothing the platform receives is ever thrown away. Kafka keeps every topic for a month; these two
tables keep the
-- records that did NOT become positions for good, so they outlive Kafka's retention.

-- Dead letters: records a consumer (listener or Kafka Streams) could not handle and parked in
gps.dead-letters.
-- Rows are never deleted: a replay or a discard only changes the status, so the history of every
record stays.
CREATE TABLE dead_letter (
    id                      bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    source                  varchar(20)  NOT NULL,
    original_topic          varchar(250),
    original_partition      integer,
    original_offset         bigint,
    consumer_group          varchar(250),
    record_key              varchar(250),
    record_value            text,
    exception_class         varchar(500),
    exception_message       text,
    stacktrace              text,
    parked_at               timestamptz  NOT NULL,
    dead_letter_partition   integer      NOT NULL,
    dead_letter_offset      bigint       NOT NULL,
    status                  varchar(10)  NOT NULL DEFAULT 'PARKED',
    replay_count            integer      NOT NULL DEFAULT 0,
    last_replayed_at        timestamptz,
    resolution_note         text,
    archived_at             timestamptz  NOT NULL DEFAULT now(),
    CONSTRAINT dead_letter_status_values CHECK (status IN ('PARKED', 'REPLAYED', 'DISCARDED')),
    -- the same dead letter delivered twice is archived once (offset + time: unique even if the topic
is ever recreated)
    CONSTRAINT dead_letter_archived_once UNIQUE (dead_letter_partition, dead_letter_offset, parked_at)
);

CREATE INDEX dead_letter_status_time ON dead_letter (status, parked_at DESC);
CREATE INDEX dead_letter_topic_time  ON dead_letter (original_topic, parked_at DESC);
CREATE INDEX dead_letter_key         ON dead_letter (record_key);

-- Rejected frames: frames the normalizer refused (bad coordinates, duplicates, stale ...), with the
reason and the
-- payload exactly as it arrived. Until now they only reached the service log.
CREATE TABLE rejected_frame (
    id                bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei              varchar(50),
    rejection_reason  text         NOT NULL,
    original_payload  text         NOT NULL,
    rejected_at       timestamptz  NOT NULL,
    payload_hash      char(32)     NOT NULL,
    archived_at       timestamptz  NOT NULL DEFAULT now(),
    -- a redelivered rejection (same payload, same moment) is stored once
    CONSTRAINT rejected_frame_archived_once UNIQUE (rejected_at, payload_hash)
);

CREATE INDEX rejected_frame_time      ON rejected_frame (rejected_at DESC);
CREATE INDEX rejected_frame_imei_time ON rejected_frame (imei, rejected_at DESC);
```

### 14.1.7 V7__connection_setting.sql

```
-- Addresses of the service's outside connections, changed from the console (Developer -> Connections)
while the
```

```
-- service runs. A missing row means the default from application.yml (gps.connections.*,
spring.data.redis.*).
--   redis     redis://host:port/database   the Redis every consumer keeps its state in (this laptop
or the HP)
--   valhalla  http://host:port             the Valhalla routing engine (all-India map on the HP home
server)
-- Kept in PostgreSQL, not in Redis: the Redis address cannot live in the Redis it points to.
CREATE TABLE connection_setting (
    name        varchar(40)   PRIMARY KEY,
    url         varchar(500)  NOT NULL,
    updated_at  timestamptz   NOT NULL DEFAULT now()
);
```

### 14.1.8 V8__vehicle_day_summary_and_report_setting.sql

```
-- Reports (Reports section of the console): one row per vehicle per day, so a month for 150 vehicles
is about 4,650
-- small rows instead of millions of positions. Rebuilt from gps_position, gps_stop, gps_trip and
vehicle_event for
-- today and yesterday every few minutes (DaySummaryJob), for older days at start and on demand.
--
-- generation 0 is the live row, recomputed from those tables. "Clear database" first freezes the live
rows into a new
-- generation (> 0) and then empties the tables, so the day's numbers survive the clear; a report adds
the generations
-- of a day up.
CREATE TABLE vehicle_day_summary (
    day                      date              NOT NULL,
    imei                     varchar(50)       NOT NULL,
    generation               integer           NOT NULL DEFAULT 0,
    tenant_code              varchar(40),
    vehicle_registration_no  varchar(50),
    readings                 integer           NOT NULL,
    first_seen_at            timestamptz,
    last_seen_at             timestamptz,
    distance_m               double precision  NOT NULL,
    moving_s                 bigint            NOT NULL,
    stopped_s                bigint            NOT NULL,
    max_speed_kmph           integer           NOT NULL,
    -- kilometres driven above economical_speed_kmph (the cost assumption in force when the row was
computed)
    fast_distance_m          double precision  NOT NULL,
    economical_speed_kmph    integer           NOT NULL,
    -- IDLING episodes inside the day: standing with the engine on longer than the idling limit
    idle_s                   bigint            NOT NULL,
    stops                    integer           NOT NULL,
    stop_s                   bigint            NOT NULL,
    engine_on_stops          integer           NOT NULL,
    trips                    integer           NOT NULL,
    overspeed_events         integer           NOT NULL,
    harsh_events             integer           NOT NULL,
    computed_at              timestamptz       NOT NULL DEFAULT now(),
    PRIMARY KEY (day, imei, generation)
);

CREATE INDEX vehicle_day_summary_vehicle ON vehicle_day_summary (imei, day);

-- Report assumptions (fuel price, mileage, idle burn ...) edited on Reports -> Cost assumptions. A
missing row means the
-- default in FuelAssumptions.defaults(). Not touched by "Clear database".
CREATE TABLE report_setting (
    name        varchar(60)   PRIMARY KEY,
    value       varchar(100)  NOT NULL,
    updated_at  timestamptz   NOT NULL DEFAULT now()
);

-- The summaries read one day of positions for every vehicle at a time.
CREATE INDEX gps_position_recorded_at ON gps_position (recorded_at);
```

### 14.1.9 V9__transport_management.sql

```
-- The transport management layer (TMS) on top of the GPS platform. Three kinds of company use it:
--   GPS provider (us)  : fits the devices, maps device <-> vehicle, sees everything (role DEVELOPER)
--   transporter        : owns or hires the trucks, uploads the shipments it carries (role
TRANSPORTER)
```

```
--   consigner          : sends the goods (steel ...) and follows its shipments to the consignee (role
CONSIGNER)
-- The consignee (who receives the goods) is a name on the shipment, not a login.

-- 1. What kind of company a tenant is. The tenants seeded from the GPS pings are truck owners:
transporters.
--    external_code: the company's number in the consigner's TMS (the transporter's vendor code, the
consignor id).
ALTER TABLE tenant
    ADD COLUMN kind          varchar(20) NOT NULL DEFAULT 'TRANSPORTER',
    ADD COLUMN external_code varchar(60),
    ADD CONSTRAINT tenant_kind_values CHECK (kind IN ('GPS_PROVIDER', 'TRANSPORTER', 'CONSIGNER'));

CREATE UNIQUE INDEX tenant_kind_external_code ON tenant (kind, external_code) WHERE external_code IS
NOT NULL;

-- 2. Who may sign in, and as what. The sign-in page asks for the role as well as user id and
password, so a user id
--    is unique per role. A transporter or consigner user belongs to exactly one company and only sees
its data.
--    password_hash: PBKDF2-HMAC-SHA256 (PasswordHashing), never the password.
CREATE TABLE app_user (
    id             uuid         PRIMARY KEY DEFAULT gen_random_uuid(),
    role           varchar(20)  NOT NULL,
    username       varchar(80)  NOT NULL,
    display_name   varchar(120) NOT NULL,
    tenant_id      uuid         REFERENCES tenant (id),
    password_hash  varchar(200) NOT NULL,
    active         boolean      NOT NULL DEFAULT true,
    created_at     timestamptz  NOT NULL DEFAULT now(),
    last_login_at  timestamptz,
    CONSTRAINT app_user_role_values CHECK (role IN ('DEVELOPER', 'TRANSPORTER', 'CONSIGNER')),
    CONSTRAINT app_user_company_for_business_roles CHECK (role = 'DEVELOPER' OR tenant_id IS NOT NULL)
);

CREATE UNIQUE INDEX app_user_role_username ON app_user (role, lower(username));

-- The key that signs sign-in tokens (PortalTokens). Made once, here, so it survives restarts and
never sits in a
-- config file; replacing it signs everyone out.
CREATE TABLE auth_key (
    id          smallint     PRIMARY KEY,
    secret      varchar(128) NOT NULL,
    created_at  timestamptz  NOT NULL DEFAULT now(),
    CONSTRAINT auth_key_single_row CHECK (id = 1)
);

INSERT INTO auth_key (id, secret) VALUES (1, replace(gen_random_uuid()::text ||
gen_random_uuid()::text, '-', ''));

-- 3. Places: the geofences a shipment starts in and ends in. code is the normalised name the shipment
files use
--    (s_org_node_name / s_dest_node_name: "ADITYAPUR"), so an uploaded row finds its fence by name.
CREATE TABLE place (
    id          bigint           GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    code        varchar(255)     NOT NULL,
    name        varchar(255)     NOT NULL,
    role        varchar(12)      NOT NULL,
    latitude    double precision NOT NULL,
    longitude   double precision NOT NULL,
    radius_m    integer          NOT NULL,
    source      varchar(20)      NOT NULL DEFAULT 'MANUAL',
    active      boolean          NOT NULL DEFAULT true,
    created_at  timestamptz      NOT NULL DEFAULT now(),
    updated_at  timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT place_once_per_role UNIQUE (code, role),
    CONSTRAINT place_role_values CHECK (role IN ('ORIGIN', 'DESTINATION')),
    CONSTRAINT place_position_valid CHECK (latitude BETWEEN -90 AND 90 AND longitude BETWEEN -180 AND
180),
    CONSTRAINT place_radius_sensible CHECK (radius_m BETWEEN 50 AND 50000)
);

-- The geofences of the smart_truck TMS (Tata Steel Jamshedpur and the destinations its trips go to).
INSERT INTO place (code, name, role, latitude, longitude, radius_m, source) VALUES
    ('JAMSHEDPUR', 'Tata Steel Jamshedpur', 'ORIGIN', 22.78859800, 86.19960000, 10000, 'SMART_TRUCK'),
    ('ADITYAPUR', 'ADITYAPUR', 'DESTINATION', 22.79630800, 86.13767333, 1153, 'SMART_TRUCK'),
    ('AGRA CITY', 'AGRA CITY', 'DESTINATION', 27.23498400, 77.85698900, 1000, 'SMART_TRUCK'),
    ('AHMEDABAD', 'AHMEDABAD', 'DESTINATION', 22.90755600, 72.42872960, 1000, 'SMART_TRUCK'),
```

```
    ('AMTA', 'AMTA', 'DESTINATION', 22.57101600, 88.03771200, 1000, 'SMART_TRUCK'),
    ('ARRAH', 'ARRAH', 'DESTINATION', 25.58772000, 84.73854200, 1000, 'SMART_TRUCK'),
    ('BALLABGARH', 'BALLABGARH', 'DESTINATION', 28.30633700, 77.30855100, 1000, 'SMART_TRUCK'),
    ('BANGALORE CITY', 'BANGALORE CITY', 'DESTINATION', 13.00517000, 77.66321600, 1000,
'SMART_TRUCK'),
    ('BEEKAY STEEL-JSR', 'BEEKAY STEEL-JSR', 'DESTINATION', 22.82696167, 86.07743500, 1000,
'SMART_TRUCK'),
    ('BEGUNIA', 'BEGUNIA', 'DESTINATION', 20.19000000, 85.42000000, 20000, 'SMART_TRUCK'),
    ('BHARATPUR', 'BHARATPUR', 'DESTINATION', 27.22711100, 77.50654200, 1000, 'SMART_TRUCK'),
    ('BHUBANESWAR', 'BHUBANESWAR', 'DESTINATION', 20.31364900, 85.85586600, 1918, 'SMART_TRUCK'),
    ('BIDADI', 'BIDADI', 'DESTINATION', 12.79750000, 77.38380000, 20000, 'SMART_TRUCK'),
    ('BMW-GAMHARIA', 'BMW-GAMHARIA', 'DESTINATION', 22.82618900, 86.07717300, 1000, 'SMART_TRUCK'),
    ('BMW-MANIFIT-JAMSHPUR', 'BMW-MANIFIT-JAMSHPUR', 'DESTINATION', 22.77848700, 86.21704000, 1000,
'SMART_TRUCK'),
    ('BOKARO', 'BOKARO', 'DESTINATION', 23.57552300, 86.20327200, 1303, 'SMART_TRUCK'),
    ('BOKARO STEEL CY', 'BOKARO STEEL CY', 'DESTINATION', 23.64020200, 86.08768000, 1000,
'SMART_TRUCK'),
    ('CALCUTTA', 'CALCUTTA', 'DESTINATION', 22.59887900, 88.22817700, 1000, 'SMART_TRUCK'),
    ('CHANDAULI MAJH', 'CHANDAULI MAJH', 'DESTINATION', 25.24578900, 83.06272900, 1000,
'SMART_TRUCK'),
    ('CHENNAI', 'CHENNAI', 'DESTINATION', 13.08270000, 80.27070000, 20000, 'SMART_TRUCK'),
    ('CRM BARA', 'CRM BARA', 'DESTINATION', 22.80624200, 86.23228300, 1000, 'SMART_TRUCK'),
    ('DANKUNI', 'DANKUNI', 'DESTINATION', 22.69511000, 88.29842100, 1000, 'SMART_TRUCK'),
    ('DHANBAD', 'DHANBAD', 'DESTINATION', 23.76292000, 86.58327800, 1000, 'SMART_TRUCK'),
    ('DHENKANAL', 'Tata Steel Meramandali', 'DESTINATION', 20.79605300, 85.26038200, 10000,
'SMART_TRUCK'),
    ('DUBURI', 'Tata Steel Kalinganagar', 'DESTINATION', 20.97041100, 86.01521100, 10000,
'SMART_TRUCK'),
    ('FARIDABAD', 'FARIDABAD', 'DESTINATION', 28.40890000, 77.31780000, 20000, 'SMART_TRUCK'),
    ('GAMHARIA', 'Tata Steel Gamharia', 'DESTINATION', 22.81289500, 86.08402200, 10000,
'SMART_TRUCK'),
    ('GAMHARIA-SANFAB', 'GAMHARIA-SANFAB', 'DESTINATION', 22.81002700, 86.11185800, 1000,
'SMART_TRUCK'),
    ('GAMHARIA-TSG', 'GAMHARIA-TSG', 'DESTINATION', 22.82371100, 86.08977800, 1000, 'SMART_TRUCK'),
    ('GAUHATI', 'GAUHATI', 'DESTINATION', 26.11693700, 91.70520900, 1000, 'SMART_TRUCK'),
    ('GIRIDIH', 'GIRIDIH', 'DESTINATION', 24.05041700, 86.42737800, 1000, 'SMART_TRUCK'),
    ('GORAKHPUR CANTT', 'GORAKHPUR CANTT', 'DESTINATION', 26.76060000, 83.37320000, 20000,
'SMART_TRUCK'),
    ('GOVINDPUR', 'GOVINDPUR', 'DESTINATION', 23.84132700, 86.51949700, 1000, 'SMART_TRUCK'),
    ('GUMMIDIPUNDI', 'GUMMIDIPUNDI', 'DESTINATION', 13.40760000, 80.10870000, 20000, 'SMART_TRUCK'),
    ('HOSUR', 'HOSUR', 'DESTINATION', 12.74090000, 77.82530000, 20000, 'SMART_TRUCK'),
    ('HOWRAH', 'HOWRAH', 'DESTINATION', 22.59824700, 88.22685300, 1000, 'SMART_TRUCK'),
    ('HUBLI', 'HUBLI', 'DESTINATION', 15.36470000, 75.12400000, 20000, 'SMART_TRUCK'),
    ('HYDERABAD', 'HYDERABAD', 'DESTINATION', 17.38500000, 78.48670000, 20000, 'SMART_TRUCK'),
    ('INDORE', 'INDORE', 'DESTINATION', 22.76177600, 75.95973300, 5000, 'SMART_TRUCK'),
    ('ISWP-JAMSHEDPUR', 'ISWP-JAMSHEDPUR', 'DESTINATION', 22.77223800, 86.23353600, 1000,
'SMART_TRUCK'),
    ('JABALPUR', 'JABALPUR', 'DESTINATION', 23.18150000, 79.98640000, 20000, 'SMART_TRUCK'),
    ('JAIPUR', 'JAIPUR', 'DESTINATION', 26.94254400, 75.75944000, 5000, 'SMART_TRUCK'),
    ('JAJPUR ROAD', 'Tata Steel Kalinganagar', 'DESTINATION', 20.97041100, 86.01521100, 10000,
'SMART_TRUCK'),
    ('JAMSHEDPUR', 'JAMSHEDPUR', 'DESTINATION', 22.77360700, 86.21681500, 1000, 'SMART_TRUCK'),
    ('JAMSHEDPUR (IBMD)', 'JAMSHEDPUR (IBMD)', 'DESTINATION', 22.82080300, 86.15939300, 1000,
'SMART_TRUCK'),
    ('JCAPCPL_JSR WORKS', 'JCAPCPL_JSR Works', 'DESTINATION', 22.78300600, 86.20552100, 1161,
'SMART_TRUCK'),
    ('JSR WORKS PLANTTRY', 'JSR WORKS PLANTTRY', 'DESTINATION', 22.78452900, 86.20697500, 1000,
'SMART_TRUCK'),
    ('KANHAN', 'KANHAN', 'DESTINATION', 21.21852900, 79.24661700, 1000, 'SMART_TRUCK'),
    ('KANPUR', 'KANPUR', 'DESTINATION', 26.39943600, 80.24158000, 5000, 'SMART_TRUCK'),
    ('KARANJIA', 'KARANJIA', 'DESTINATION', 21.75365600, 85.97498700, 1000, 'SMART_TRUCK'),
    ('KARUR %SR#', 'KARUR %SR#', 'DESTINATION', 10.94527000, 78.12932400, 1000, 'SMART_TRUCK'),
    ('KHOPOLI', 'KHOPOLI', 'DESTINATION', 18.80107700, 73.28833700, 1000, 'SMART_TRUCK'),
    ('KOLABIRA', 'KOLABIRA', 'DESTINATION', 22.80395600, 86.03067600, 1000, 'SMART_TRUCK'),
    ('LUDHIANA', 'LUDHIANA', 'DESTINATION', 30.87868000, 75.98849800, 5000, 'SMART_TRUCK'),
    ('MALDA TOWN', 'MALDA TOWN', 'DESTINATION', 25.02348200, 88.16378700, 1000, 'SMART_TRUCK'),
    ('MALLAVALLI', 'MALLAVALLI', 'DESTINATION', 16.69823800, 80.89031600, 1000, 'SMART_TRUCK'),
    ('MEDAK', 'MEDAK', 'DESTINATION', 17.66734300, 78.18116400, 1000, 'SMART_TRUCK'),
    ('MEDCHAL', 'MEDCHAL', 'DESTINATION', 17.62970000, 78.48090000, 20000, 'SMART_TRUCK'),
    ('NANDED', 'NANDED', 'DESTINATION', 19.13152700, 77.33448900, 1000, 'SMART_TRUCK'),
    ('PALWAL', 'PALWAL', 'DESTINATION', 28.20680000, 77.30710000, 2124, 'SMART_TRUCK'),
    ('PATNA CITY', 'PATNA CITY', 'DESTINATION', 25.59410000, 85.13760000, 20000, 'SMART_TRUCK'),
    ('PILKHUWA', 'PILKHUWA', 'DESTINATION', 28.69986100, 77.58792300, 1000, 'SMART_TRUCK'),
    ('PITHAMPUR (M.P.)', 'PITHAMPUR (M.P.)', 'DESTINATION', 22.60760000, 75.67990000, 20000,
'SMART_TRUCK'),
    ('PURNEA JN.', 'PURNEA JN.', 'DESTINATION', 25.72052100, 87.51296500, 1000, 'SMART_TRUCK'),
    ('RAJKOT', 'RAJKOT', 'DESTINATION', 22.37897100, 70.92816900, 1000, 'SMART_TRUCK'),
    ('RAMGARH CANTT', 'RAMGARH CANTT', 'DESTINATION', 23.68904800, 85.50720900, 1000, 'SMART_TRUCK'),
```

```
    ('RANCHI', 'RANCHI', 'DESTINATION', 23.26619600, 85.30533000, 1000, 'SMART_TRUCK'),
    ('RANIGANJ', 'RANIGANJ', 'DESTINATION', 23.63251100, 87.12493200, 1000, 'SMART_TRUCK'),
    ('RANIPPETTAI', 'RANIPPETTAI', 'DESTINATION', 13.00416800, 79.30485100, 1000, 'SMART_TRUCK'),
    ('RANJANGAON', 'RANJANGAON', 'DESTINATION', 18.77260000, 74.24820000, 20000, 'SMART_TRUCK'),
    ('ROURKELA', 'ROURKELA', 'DESTINATION', 22.20408400, 84.74773400, 1000, 'SMART_TRUCK'),
    ('RUDRAPUR', 'RUDRAPUR', 'DESTINATION', 28.98450000, 79.41410000, 20000, 'SMART_TRUCK'),
    ('SANAND', 'SANAND', 'DESTINATION', 22.99220000, 72.38190000, 20000, 'SMART_TRUCK'),
    ('SANGAREDDY', 'SANGAREDDY', 'DESTINATION', 17.62480000, 78.08660000, 20000, 'SMART_TRUCK'),
    ('SANSWADI', 'SANSWADI', 'DESTINATION', 18.68000000, 74.05000000, 20000, 'SMART_TRUCK'),
    ('SARAIKELA KHARSAWAN', 'SARAIKELA KHARSAWAN', 'DESTINATION', 22.79090400, 86.15172400, 1000,
'SMART_TRUCK'),
    ('SINGUR', 'SINGUR', 'DESTINATION', 22.81753300, 88.24571600, 1000, 'SMART_TRUCK'),
    ('SURAT', 'SURAT', 'DESTINATION', 21.09586700, 72.88758700, 1000, 'SMART_TRUCK'),
    ('TANGI', 'TANGI', 'DESTINATION', 20.58118300, 86.02957900, 1000, 'SMART_TRUCK'),
    ('TBL- BARA AGRICO', 'TBL- BARA AGRICO', 'DESTINATION', 22.81376500, 86.23905800, 1000,
'SMART_TRUCK'),
    ('TCIL-JAMSHEDPUR', 'TCIL-JAMSHEDPUR', 'DESTINATION', 22.78957600, 86.22433800, 1000,
'SMART_TRUCK'),
    ('TIRUNINRAVUR', 'TIRUNINRAVUR', 'DESTINATION', 13.12533000, 80.01223000, 1000, 'SMART_TRUCK'),
    ('TITAGARH', 'TITAGARH', 'DESTINATION', 22.74103900, 88.37270800, 1000, 'SMART_TRUCK'),
    ('TRYL-BARA', 'TRYL-BARA', 'DESTINATION', 22.81165800, 86.23227600, 1000, 'SMART_TRUCK'),
    ('UDHAM SINGH NAGAR', 'UDHAM SINGH NAGAR', 'DESTINATION', 28.97500000, 79.40000000, 20000,
'SMART_TRUCK'),
    ('VADODARA', 'VADODARA', 'DESTINATION', 22.30720000, 73.18120000, 20000, 'SMART_TRUCK'),
    ('VIRAMGAM', 'VIRAMGAM', 'DESTINATION', 23.08149760, 72.17208500, 1000, 'SMART_TRUCK')
ON CONFLICT DO NOTHING;

-- 4. Every shipment file uploaded (Excel, CSV, pasted rows, or the smart_truck trips), with what it
did. Uploading the
--    same file again is harmless: its rows are found unchanged (row_hash) and nothing is written
twice.
CREATE TABLE shipment_import (
    id              bigint       GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    file_name       varchar(255) NOT NULL,
    file_sha256     varchar(64)  NOT NULL,
    format          varchar(12)  NOT NULL,
    uploaded_by     varchar(80)  NOT NULL,
    uploaded_role   varchar(20)  NOT NULL,
    tenant_id       uuid         REFERENCES tenant (id),
    rows_total      integer      NOT NULL,
    rows_created    integer      NOT NULL,
    rows_updated    integer      NOT NULL,
    rows_unchanged  integer      NOT NULL,
    rows_failed     integer      NOT NULL,
    problems        jsonb        NOT NULL DEFAULT '[]'::jsonb,
    column_mapping  jsonb        NOT NULL DEFAULT '{}'::jsonb,
    uploaded_at     timestamptz  NOT NULL DEFAULT now()
);

CREATE INDEX shipment_import_file ON shipment_import (file_sha256);
CREATE INDEX shipment_import_recent ON shipment_import (uploaded_at DESC);

-- 5. A shipment: one trip of one truck carrying one consigner's goods, as the transporter's file
describes it
--    (the smart_truck tta_trips layout), plus what the GPS says about it.
--
--    Identity: the trip number within the transporter. A row uploaded again (the same file twice, a
corrected file,
--    the consigner's next export) updates its shipment instead of adding one - "MULTIPLE UPLOAD"
can't duplicate.
--
--    The GPS columns are written by the shipment-tracking module, always as the EARLIEST reading that
proves them
--    (departure = first reading outside the origin fence after the gate-out, arrival = first reading
inside the
--    destination fence after being outside it). A reading delivered twice, late (device memory) or
replayed can
--    therefore only confirm or move a moment earlier, never invent a second one.
CREATE TABLE shipment (
    id                      bigint           GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    trip_no                 varchar(60)      NOT NULL,
    source                  varchar(12)      NOT NULL,
    transporter_tenant_id   uuid             NOT NULL REFERENCES tenant (id),
    transporter_code        varchar(60),
    consigner_tenant_id     uuid             REFERENCES tenant (id),
    consigner_code          varchar(60),
    consigner_unit          varchar(255),
    consignee_name          varchar(255),
```

```
    consignee_address        varchar(500),
    vehicle_registration_no  varchar(50)      NOT NULL,
    device_imei              varchar(50),
    vehicle_type             varchar(100),
    trip_class               varchar(20),
    trip_type                varchar(100),
    origin_name              varchar(255),
    destination_name         varchar(255),
    load_plant               varchar(255),
    origin_place_id          bigint           REFERENCES place (id),
    destination_place_id     bigint           REFERENCES place (id),
    booked_at                timestamptz,
    dispatched_at            timestamptz      NOT NULL,
    eta_at                   timestamptz,
    reported_ata_at          timestamptz,
    reported_end_at          timestamptz,
    invoice_no               varchar(100),
    shipment_ref             varchar(100),
    gate_entry_no            varchar(100),
    card_id                  varchar(50),
    route_id                 varchar(30),
    material                 varchar(500),
    quantity                 numeric(14, 2),
    driver_name              varchar(255),
    driver_mobile            varchar(20),
    source_status            varchar(30),
    close_reason             varchar(255),
    left_origin_at           timestamptz,
    outside_destination_at   timestamptz,
    arrived_at               timestamptz,
    last_reading_at          timestamptz,
    last_latitude            double precision,
    last_longitude           double precision,
    last_speed_kmph          integer,
    closed_at                timestamptz,
    closed_reason            varchar(60),
    row_hash                 varchar(64)      NOT NULL,
    import_id                bigint           REFERENCES shipment_import (id),
    created_at               timestamptz      NOT NULL DEFAULT now(),
    updated_at               timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT shipment_trip_once_per_transporter UNIQUE (transporter_tenant_id, trip_no)
);

-- the tracking module looks up a reading's shipments by vehicle (or by the IMEI in the file) and time
CREATE INDEX shipment_vehicle_dispatch ON shipment (vehicle_registration_no, dispatched_at DESC);
CREATE INDEX shipment_device_dispatch ON shipment (device_imei, dispatched_at DESC);
CREATE INDEX shipment_consigner ON shipment (consigner_tenant_id, dispatched_at DESC);
CREATE INDEX shipment_transporter ON shipment (transporter_tenant_id, dispatched_at DESC);

-- portal alert lists join events to the shipment windows by vehicle
CREATE INDEX vehicle_event_vehicle_time ON vehicle_event (vehicle_registration_no, started_at DESC);
```

### 14.1.10 V10__geofencing.sql

```
-- Geofencing module (consumer group gps-geofence-service): the place table becomes the geofence
master, and every
-- visit of every vehicle to every fence is kept, for the live board and the day-wise geofence report.

-- 1. place = geofence: a circle or a polygon, more roles, its own dwell and speed limits, and where
it came from.
ALTER TABLE place DROP CONSTRAINT place_role_values;
ALTER TABLE place DROP CONSTRAINT place_radius_sensible;
ALTER TABLE place
    ADD COLUMN shape             varchar(10) NOT NULL DEFAULT 'CIRCLE',
    -- POLYGON: [[latitude, longitude], ...] in order around the border; radius_m is then the covering
radius
    ADD COLUMN polygon           jsonb,
    -- inside longer than this raises GEOFENCE_DWELL; null = the role's default
(gps.modules.geofence.dwell-*), 0 = never
    ADD COLUMN dwell_alert_min   integer,
    -- the speed limit inside the fence (a plant's 20 km/h); null = the vehicle's own rule only
    ADD COLUMN speed_limit_kmph  integer,
    -- smart_truck's id (smart_truck:{i_fence_id}) for fences copied from it
    ADD COLUMN external_ref      varchar(60),
    -- how smart_truck derived it: plant (surveyed), anchor (medoid of pings at the facility),
anchor_unverified,
```

```
    -- gazetteer (a town centre standing in for an unknown customer site: a "far" reading may still be
a delivery)
    ADD COLUMN source_detail     varchar(30),
    ADD COLUMN trips_seen        integer,
    ADD COLUMN spread_m          integer,
    ADD CONSTRAINT place_role_values CHECK (role IN ('ORIGIN', 'DESTINATION', 'YARD', 'CHECKPOINT',
'ZONE')),
    ADD CONSTRAINT place_shape_values CHECK (shape IN ('CIRCLE', 'POLYGON')),
    ADD CONSTRAINT place_radius_sensible CHECK (shape = 'POLYGON' OR radius_m BETWEEN 50 AND 50000),
    ADD CONSTRAINT place_polygon_present CHECK (shape = 'CIRCLE' OR (jsonb_typeof(polygon) = 'array'
AND jsonb_array_length(polygon) >= 3)),
    ADD CONSTRAINT place_dwell_sensible CHECK (dwell_alert_min IS NULL OR dwell_alert_min >= 0),
    ADD CONSTRAINT place_speed_sensible CHECK (speed_limit_kmph IS NULL OR speed_limit_kmph BETWEEN 5
AND 150);

-- 2. The geofence master data of smart_truck (table geofences, 87 fences, copied 2026-11-02). A fence
edited here
--    (source MANUAL) is never overwritten; POST /api/geofences/import/smart-truck copies newer
smart_truck data later.
INSERT INTO place (code, name, role, latitude, longitude, radius_m, source, active, external_ref,
source_detail, trips_seen, spread_m)
SELECT g.code, g.name, g.role, g.lat, g.lon, g.radius, 'SMART_TRUCK', g.active, 'smart_truck:' ||
g.fence_id, g.source_detail,
       g.trips, g.spread
FROM (VALUES
    (1, 'ADITYAPUR', 'ADITYAPUR', 'DESTINATION', 22.79630800, 86.13767333, 1153, 34, 769, false, true,
'anchor'),
    (2, 'AGRA CITY', 'AGRA CITY', 'DESTINATION', 27.23498400, 77.85698900, 1000, 4, 74, false, true,
'anchor'),
    (3, 'AHMEDABAD', 'AHMEDABAD', 'DESTINATION', 22.90755600, 72.42872960, 1000, 3, 144, false, true,
'anchor'),
    (4, 'AMTA', 'AMTA', 'DESTINATION', 22.57101600, 88.03771200, 1000, 17, 337, false, true,
'anchor'),
    (5, 'ARRAH', 'ARRAH', 'DESTINATION', 25.58772000, 84.73854200, 1000, 4, 66, false, true,
'anchor_unverified'),
    (6, 'BALLABGARH', 'BALLABGARH', 'DESTINATION', 28.30633700, 77.30855100, 1000, 24, 625, false,
true, 'anchor'),
    (7, 'BANGALORE CITY', 'BANGALORE CITY', 'DESTINATION', 13.00517000, 77.66321600, 1000, 23, 189,
false, true, 'anchor'),
    (8, 'BEEKAY STEEL-JSR', 'BEEKAY STEEL-JSR', 'DESTINATION', 22.82696167, 86.07743500, 1000, 24, 61,
false, true, 'anchor_unverified'),
    (9, 'BEGUNIA', 'BEGUNIA', 'DESTINATION', 20.19000000, 85.42000000, 20000, 2, 295294, false, true,
'gazetteer'),
    (10, 'BHARATPUR', 'BHARATPUR', 'DESTINATION', 27.22711100, 77.50654200, 1000, 2, 522, false, true,
'anchor'),
    (11, 'BHUBANESWAR', 'BHUBANESWAR', 'DESTINATION', 20.31364900, 85.85586600, 1918, 23, 1279, false,
true, 'anchor'),
    (12, 'BIDADI', 'BIDADI', 'DESTINATION', 12.79750000, 77.38380000, 20000, 4, 3765, false, true,
'gazetteer'),
    (13, 'BMW-GAMHARIA', 'BMW-GAMHARIA', 'DESTINATION', 22.82618900, 86.07717300, 1000, 5, 77, false,
true, 'anchor'),
    (14, 'BMW-MANIFIT-JAMSHPUR', 'BMW-MANIFIT-JAMSHPUR', 'DESTINATION', 22.77848700, 86.21704000,
1000, 2, 52, false, true, 'anchor_unverified'),
    (15, 'BOKARO', 'BOKARO', 'DESTINATION', 23.57552300, 86.20327200, 1303, 3, 869, false, true,
'anchor'),
    (16, 'BOKARO STEEL CY', 'BOKARO STEEL CY', 'DESTINATION', 23.64020200, 86.08768000, 1000, 2, 207,
false, true, 'anchor'),
    (17, 'CALCUTTA', 'CALCUTTA', 'DESTINATION', 22.59887900, 88.22817700, 1000, 2, 201, false, true,
'anchor'),
    (18, 'CHANDAULI MAJH', 'CHANDAULI MAJH', 'DESTINATION', 25.24578900, 83.06272900, 1000, 7, 109,
false, true, 'anchor'),
    (19, 'CHENNAI', 'CHENNAI', 'DESTINATION', 13.08270000, 80.27070000, 20000, 16, 599, false, true,
'gazetteer'),
    (20, 'CRM BARA', 'CRM BARA', 'DESTINATION', 22.80624200, 86.23228300, 1000, 64, 147, false, true,
'anchor_unverified'),
    (21, 'DANKUNI', 'DANKUNI', 'DESTINATION', 22.69511000, 88.29842100, 1000, 8, 140, false, true,
'anchor'),
    (22, 'DHANBAD', 'DHANBAD', 'DESTINATION', 23.76292000, 86.58327800, 1000, 5, 99, false, true,
'anchor'),
    (23, 'DHENKANAL', 'Tata Steel Meramandali', 'DESTINATION', 20.79605300, 85.26038200, 10000, 44,
204, true, true, 'plant'),
    (24, 'DUBURI', 'Tata Steel Kalinganagar', 'DESTINATION', 20.97041100, 86.01521100, 10000, 37, 486,
true, true, 'plant'),
    (25, 'FARIDABAD', 'FARIDABAD', 'DESTINATION', 28.40890000, 77.31780000, 20000, 15, 3334, false,
true, 'gazetteer'),
    (26, 'GAMHARIA', 'Tata Steel Gamharia', 'DESTINATION', 22.81289500, 86.08402200, 10000, 86, 1136,
true, true, 'plant'),
```

```
    (27, 'GAMHARIA-SANFAB', 'GAMHARIA-SANFAB', 'DESTINATION', 22.81002700, 86.11185800, 1000, 6, 241,
false, true, 'anchor'),
    (28, 'GAMHARIA-TSG', 'GAMHARIA-TSG', 'DESTINATION', 22.82371100, 86.08977800, 1000, 12, 104,
false, true, 'anchor_unverified'),
    (29, 'GAUHATI', 'GAUHATI', 'DESTINATION', 26.11693700, 91.70520900, 1000, 3, 208, false, true,
'anchor'),
    (30, 'GIRIDIH', 'GIRIDIH', 'DESTINATION', 24.05041700, 86.42737800, 1000, 34, 553, false, true,
'anchor'),
    (31, 'GORAKHPUR CANTT', 'GORAKHPUR CANTT', 'DESTINATION', 26.76060000, 83.37320000, 20000, 3, 165,
false, true, 'gazetteer'),
    (32, 'GOVINDPUR', 'GOVINDPUR', 'DESTINATION', 23.84132700, 86.51949700, 1000, 36, 256, false,
true, 'anchor'),
    (33, 'GUMMIDIPUNDI', 'GUMMIDIPUNDI', 'DESTINATION', 13.40760000, 80.10870000, 20000, 4, 15031,
false, true, 'gazetteer'),
    (34, 'HOSUR', 'HOSUR', 'DESTINATION', 12.74090000, 77.82530000, 20000, 2, 1437597, false, true,
'gazetteer'),
    (35, 'HOWRAH', 'HOWRAH', 'DESTINATION', 22.59824700, 88.22685300, 1000, 14, 492, false, true,
'anchor'),
    (36, 'HUBLI', 'HUBLI', 'DESTINATION', 15.36470000, 75.12400000, 20000, 2, 1420656, false, true,
'gazetteer'),
    (37, 'HYDERABAD', 'HYDERABAD', 'DESTINATION', 17.38500000, 78.48670000, 20000, 6, 79, false, true,
'gazetteer'),
    (38, 'INDORE', 'INDORE', 'DESTINATION', 22.76177600, 75.95973300, 5000, 3, 11950, false, true,
'anchor'),
    (39, 'ISWP-JAMSHEDPUR', 'ISWP-JAMSHEDPUR', 'DESTINATION', 22.77223800, 86.23353600, 1000, 10, 69,
false, true, 'anchor_unverified'),
    (40, 'JABALPUR', 'JABALPUR', 'DESTINATION', 23.18150000, 79.98640000, 20000, 2, 625747, false,
true, 'gazetteer'),
    (41, 'JAIPUR', 'JAIPUR', 'DESTINATION', 26.94254400, 75.75944000, 5000, 19, 6533, false, true,
'anchor'),
    (42, 'JAJPUR ROAD', 'Tata Steel Kalinganagar', 'DESTINATION', 20.97041100, 86.01521100, 10000, 23,
186, true, true, 'plant'),
    (43, 'JAMSHEDPUR', 'JAMSHEDPUR', 'DESTINATION', 22.77360700, 86.21681500, 1000, 128, 96, false,
true, 'anchor'),
    (44, 'JAMSHEDPUR', 'Tata Steel Jamshedpur', 'ORIGIN', 22.78859800, 86.19960000, 10000, 1346, 1790,
true, true, 'plant'),
    (45, 'JAMSHEDPUR (IBMD)', 'JAMSHEDPUR (IBMD)', 'DESTINATION', 22.82080300, 86.15939300, 1000, 26,
195, false, true, 'anchor'),
    (46, 'JCAPCPL_JSR WORKS', 'JCAPCPL_JSR Works', 'DESTINATION', 22.78300600, 86.20552100, 1161, 33,
774, false, true, 'anchor'),
    (47, 'JSR WORKS PLANTTRY', 'JSR WORKS PLANTTRY', 'DESTINATION', 22.78452900, 86.20697500, 1000,
20, 529, false, true, 'anchor'),
    (48, 'KANHAN', 'KANHAN', 'DESTINATION', 21.21852900, 79.24661700, 1000, 20, 502, false, true,
'anchor'),
    (49, 'KANPUR', 'KANPUR', 'DESTINATION', 26.39943600, 80.24158000, 5000, 11, 18926, false, true,
'anchor'),
    (50, 'KARANJIA', 'KARANJIA', 'DESTINATION', 21.75365600, 85.97498700, 1000, 2, 253, false, true,
'anchor'),
    (51, 'KARUR %SR#', 'KARUR %SR#', 'DESTINATION', 10.94527000, 78.12932400, 1000, 3, 270, false,
true, 'anchor'),
    (52, 'KHOPOLI', 'KHOPOLI', 'DESTINATION', 18.80107700, 73.28833700, 1000, 3, 540, false, true,
'anchor'),
    (54, 'KOLABIRA', 'KOLABIRA', 'DESTINATION', 22.80395600, 86.03067600, 1000, 4, 99, false, true,
'anchor'),
    (55, 'LUDHIANA', 'LUDHIANA', 'DESTINATION', 30.87868000, 75.98849800, 5000, 2, 6000, false, true,
'anchor'),
    (56, 'MALDA TOWN', 'MALDA TOWN', 'DESTINATION', 25.02348200, 88.16378700, 1000, 8, 470, false,
true, 'anchor'),
    (57, 'MALLAVALLI', 'MALLAVALLI', 'DESTINATION', 16.69823800, 80.89031600, 1000, 5, 506, false,
true, 'anchor'),
    (58, 'MEDAK', 'MEDAK', 'DESTINATION', 17.66734300, 78.18116400, 1000, 3, 153, false, true,
'anchor'),
    (59, 'MEDCHAL', 'MEDCHAL', 'DESTINATION', 17.62970000, 78.48090000, 20000, 4, 139, false, true,
'gazetteer'),
    (60, 'NANDED', 'NANDED', 'DESTINATION', 19.13152700, 77.33448900, 1000, 2, 247, false, true,
'anchor'),
    (61, 'PALWAL', 'PALWAL', 'DESTINATION', 28.20680000, 77.30710000, 2124, 31, 1416, false, true,
'anchor'),
    (62, 'PATNA CITY', 'PATNA CITY', 'DESTINATION', 25.59410000, 85.13760000, 20000, 58, 454, false,
true, 'gazetteer'),
    (63, 'PILKHUWA', 'PILKHUWA', 'DESTINATION', 28.69986100, 77.58792300, 1000, 9, 114, false, true,
'anchor'),
    (64, 'PITHAMPUR (M.P.)', 'PITHAMPUR (M.P.)', 'DESTINATION', 22.60760000, 75.67990000, 20000, 11,
10503, false, true, 'gazetteer'),
    (65, 'PURNEA JN.', 'PURNEA JN.', 'DESTINATION', 25.72052100, 87.51296500, 1000, 7, 251, false,
true, 'anchor'),
    (66, 'RAJKOT', 'RAJKOT', 'DESTINATION', 22.37897100, 70.92816900, 1000, 7, 640, false, true,
'anchor'),
```

```
    (67, 'RAMGARH CANTT', 'RAMGARH CANTT', 'DESTINATION', 23.68904800, 85.50720900, 1000, 2, 82,
false, true, 'anchor'),
    (68, 'RANCHI', 'RANCHI', 'DESTINATION', 23.26619600, 85.30533000, 1000, 3, 43, false, true,
'anchor'),
    (69, 'RANIGANJ', 'RANIGANJ', 'DESTINATION', 23.63251100, 87.12493200, 1000, 10, 132, false, true,
'anchor'),
    (70, 'RANIPPETTAI', 'RANIPPETTAI', 'DESTINATION', 13.00416800, 79.30485100, 1000, 3, 47, false,
true, 'anchor'),
    (71, 'RANJANGAON', 'RANJANGAON', 'DESTINATION', 18.77260000, 74.24820000, 20000, 9, 2006, false,
true, 'gazetteer'),
    (72, 'ROURKELA', 'ROURKELA', 'DESTINATION', 22.20408400, 84.74773400, 1000, 11, 149, false, true,
'anchor'),
    (73, 'RUDRAPUR', 'RUDRAPUR', 'DESTINATION', 28.98450000, 79.41410000, 20000, 27, 3020, false,
true, 'gazetteer'),
    (74, 'SANAND', 'SANAND', 'DESTINATION', 22.99220000, 72.38190000, 20000, 32, 136, false, true,
'gazetteer'),
    (75, 'SANGAREDDY', 'SANGAREDDY', 'DESTINATION', 17.62480000, 78.08660000, 20000, 2, 1022501,
false, true, 'gazetteer'),
    (76, 'SANSWADI', 'SANSWADI', 'DESTINATION', 18.68000000, 74.05000000, 20000, 22, 424, false, true,
'gazetteer'),
    (77, 'SARAIKELA KHARSAWAN', 'SARAIKELA KHARSAWAN', 'DESTINATION', 22.79090400, 86.15172400, 1000,
14, 435, false, true, 'anchor'),
    (78, 'SINGUR', 'SINGUR', 'DESTINATION', 22.81753300, 88.24571600, 1000, 2, 72, false, true,
'anchor'),
    (79, 'SURAT', 'SURAT', 'DESTINATION', 21.09586700, 72.88758700, 1000, 3, 260, false, true,
'anchor'),
    (80, 'TANGI', 'TANGI', 'DESTINATION', 20.58118300, 86.02957900, 1000, 13, 264, false, true,
'anchor'),
    (81, 'TBL- BARA AGRICO', 'TBL- BARA AGRICO', 'DESTINATION', 22.81376500, 86.23905800, 1000, 25,
36, false, true, 'anchor'),
    (82, 'TCIL-JAMSHEDPUR', 'TCIL-JAMSHEDPUR', 'DESTINATION', 22.78957600, 86.22433800, 1000, 6, 665,
false, true, 'anchor_unverified'),
    (83, 'TIRUNINRAVUR', 'TIRUNINRAVUR', 'DESTINATION', 13.12533000, 80.01223000, 1000, 22, 409,
false, true, 'anchor'),
    (84, 'TITAGARH', 'TITAGARH', 'DESTINATION', 22.74103900, 88.37270800, 1000, 4, 207, false, true,
'anchor_unverified'),
    (85, 'TRYL-BARA', 'TRYL-BARA', 'DESTINATION', 22.81165800, 86.23227600, 1000, 58, 126, false,
true, 'anchor'),
    (86, 'UDHAM SINGH NAGAR', 'UDHAM SINGH NAGAR', 'DESTINATION', 28.97500000, 79.40000000, 20000, 2,
267, false, true, 'gazetteer'),
    (88, 'VADODARA', 'VADODARA', 'DESTINATION', 22.30720000, 73.18120000, 20000, 2, 363, false, true,
'gazetteer'),
    (89, 'VIRAMGAM', 'VIRAMGAM', 'DESTINATION', 23.08149760, 72.17208500, 1000, 7, 441, false, true,
'anchor')
) AS g (fence_id, code, name, role, lat, lon, radius, trips, spread, manual, active, source_detail)
ON CONFLICT (code, role) DO UPDATE SET
    name = EXCLUDED.name, latitude = EXCLUDED.latitude, longitude = EXCLUDED.longitude, radius_m =
EXCLUDED.radius_m,
    active = EXCLUDED.active, external_ref = EXCLUDED.external_ref, source_detail =
EXCLUDED.source_detail,
    trips_seen = EXCLUDED.trips_seen, spread_m = EXCLUDED.spread_m, updated_at = now()
WHERE place.source = 'SMART_TRUCK';

-- 3. One row per visit (or pass) of one vehicle to one fence, written by the geofence module:
--      VISIT  inside long enough to be believed (15 min, the smart_truck rule); open (exited_at null)
while inside
--      PASS   through without stopping long enough, or over the fence between two readings (inferred)
--    Stamps are readings the vehicle was seen at: entered_at = first reading inside, exited_at = last
reading inside.
--    entry_gap_s / exit_gap_s are the gaps to the readings outside: the uncertainty of each stamp.
--    Identity: device + fence + kind + entry, so the same visit found twice (a redelivery, a replay)
is one row.
--    No foreign key to place: a fence deleted later keeps its history, with its code and name as they
were.
CREATE TABLE geofence_visit (
    id                      bigint          GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    imei                    varchar(50)     NOT NULL,
    vehicle_registration_no  varchar(50),
    tenant_code             varchar(40),
    place_id                bigint          NOT NULL,
    place_code              varchar(255)    NOT NULL,
    place_name              varchar(255)    NOT NULL,
    place_role              varchar(12)     NOT NULL,
    kind                    varchar(6)      NOT NULL,
    entered_at              timestamptz     NOT NULL,
    exited_at               timestamptz,
    entry_gap_s             bigint,
    exit_gap_s              bigint,
```

```
    entry_latitude          double precision NOT NULL,
    entry_longitude         double precision NOT NULL,
    exit_latitude           double precision,
    exit_longitude          double precision,
    dwell_alert_at          timestamptz,
    dwell_limit_min         integer,
    inferred                boolean         NOT NULL DEFAULT false,
    exit_reason             varchar(20),
    created_at              timestamptz     NOT NULL DEFAULT now(),
    updated_at              timestamptz     NOT NULL DEFAULT now(),
    CONSTRAINT geofence_visit_once UNIQUE (imei, place_id, kind, entered_at),
    CONSTRAINT geofence_visit_kind_values CHECK (kind IN ('VISIT', 'PASS')),
    CONSTRAINT geofence_visit_period_valid CHECK (exited_at IS NULL OR exited_at >= entered_at)
);

-- the live board: who is inside which fence now
CREATE INDEX geofence_visit_open ON geofence_visit (place_id) WHERE exited_at IS NULL;
-- the day report and the visit log: visits overlapping a period
CREATE INDEX geofence_visit_time ON geofence_visit (entered_at DESC);
CREATE INDEX geofence_visit_place_time ON geofence_visit (place_id, entered_at DESC);
CREATE INDEX geofence_visit_vehicle_time ON geofence_visit (vehicle_registration_no, entered_at DESC);
CREATE INDEX geofence_visit_imei_time ON geofence_visit (imei, entered_at DESC);
```

### 14.1.11 V11__route_plans.sql

```
-- Route deviation, live ETA and waypoints (modules route-deviation and waypoints,
docs/ROUTE_DEVIATION_ETA_DESIGN.md).
--
-- A route plan is one trip of one truck along a road route planned by Valhalla (the HP home server)
or GraphHopper
-- (this laptop): made automatically for every open shipment with an origin and a destination
geofence, or by hand
-- (Alerts -> Route deviation -> Plan a route). The modules project every reading onto the plan's
line; Redis holds the
-- per-reading state, this table the plan itself and the live numbers whenever they change.

CREATE TABLE route_plan (
    id                      bigint          GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    shipment_id             bigint          REFERENCES shipment (id) ON DELETE CASCADE,
    imei                    varchar(50),
    vehicle_registration_no  varchar(50)     NOT NULL,
    tenant_code             varchar(80),
    source                  varchar(12)     NOT NULL,
    -- ACTIVE while the truck is on its way; ARRIVED at the destination; CANCELLED by hand; CLOSED
when its shipment
    -- closed without an arrival, or a newer plan for the same truck replaced it
    status                  varchar(12)     NOT NULL DEFAULT 'ACTIVE',
    origin_name             varchar(255),
    origin_place_id         bigint          REFERENCES place (id),
    origin_lat              double precision NOT NULL,
    origin_lon              double precision NOT NULL,
    destination_name        varchar(255),
    destination_place_id    bigint          REFERENCES place (id),
    destination_lat         double precision NOT NULL,
    destination_lon         double precision NOT NULL,
    destination_radius_m    double precision NOT NULL,
    engine                  varchar(20)     NOT NULL,
    costing                 varchar(20)     NOT NULL,
    -- the current line: version 1 is the dispatch plan, each ROUTE_ALTERNATE adds one (a re-plan from
where the truck was)
    version                 integer         NOT NULL DEFAULT 1,
    reason                  varchar(20)     NOT NULL,
    shape                   text            NOT NULL,
    timings                 jsonb           NOT NULL DEFAULT '[]'::jsonb,
    length_m                double precision NOT NULL,
    planned_s               double precision NOT NULL,
    -- distance already done before the current version began
    offset_m                double precision NOT NULL DEFAULT 0,
    first_shape             text            NOT NULL,
    first_length_m          double precision NOT NULL,
    first_planned_s         double precision NOT NULL,
    -- actual / planned driving time this truck starts with: the lane's median of earlier arrived
trips, else 1
    initial_pace            double precision NOT NULL,
    planned_start_at        timestamptz     NOT NULL,
    -- live numbers, written by the route-deviation module when they change
    route_status            varchar(12)     NOT NULL DEFAULT 'WAITING',
```

```
    departed_at             timestamptz,
    promised_eta_at         timestamptz,
    eta_at                  timestamptz,
    arrived_at              timestamptz,
    pace                    double precision,
    chainage_m              double precision,
    cross_track_m           double precision,
    last_reading_at         timestamptz,
    last_lat                double precision,
    last_lon                double precision,
    deviations              integer          NOT NULL DEFAULT 0,
    deviation_km            double precision NOT NULL DEFAULT 0,
    deviation_minutes       integer          NOT NULL DEFAULT 0,
    wrong_ways              integer          NOT NULL DEFAULT 0,
    alternates              integer          NOT NULL DEFAULT 0,
    created_by              varchar(80),
    created_at              timestamptz      NOT NULL DEFAULT now(),
    updated_at              timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT route_plan_status_values CHECK (status IN ('ACTIVE', 'ARRIVED', 'CANCELLED',
'CLOSED')),
    CONSTRAINT route_plan_source_values CHECK (source IN ('SHIPMENT', 'MANUAL')),
    CONSTRAINT route_plan_route_status_values CHECK (route_status IN ('WAITING', 'ON_ROUTE',
'SUSPECT', 'OFF_ROUTE', 'ARRIVED')),
    CONSTRAINT route_plan_positive CHECK (length_m > 0 AND planned_s > 0 AND destination_radius_m > 0)
);

-- a shipment is planned once; the modules find a truck's plan by vehicle or device
CREATE UNIQUE INDEX route_plan_one_per_shipment ON route_plan (shipment_id) WHERE shipment_id IS NOT
NULL;
CREATE INDEX route_plan_active_vehicle ON route_plan (vehicle_registration_no) WHERE status =
'ACTIVE';
CREATE INDEX route_plan_active_imei ON route_plan (imei) WHERE status = 'ACTIVE';
CREATE INDEX route_plan_recent ON route_plan (planned_start_at DESC);
-- the lane's pace history (median of arrived trips between the same two geofences)
CREATE INDEX route_plan_lane ON route_plan (origin_place_id, destination_place_id) WHERE status =
'ARRIVED';

-- The stops on the way: waypoints planned by hand (MANUAL) and the geofences the planned line crosses
(FENCE).
CREATE TABLE route_stop (
    id              bigint           GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    plan_id         bigint           NOT NULL REFERENCES route_plan (id) ON DELETE CASCADE,
    seq             integer          NOT NULL,
    kind            varchar(10)      NOT NULL,
    place_id        bigint           REFERENCES place (id),
    name            varchar(255)     NOT NULL,
    latitude        double precision NOT NULL,
    longitude       double precision NOT NULL,
    radius_m        double precision NOT NULL,
    -- distance along the current line; -1 when the current line does not pass it
    chainage_m      double precision NOT NULL,
    stay_min        integer          NOT NULL DEFAULT 0,
    status          varchar(10)      NOT NULL DEFAULT 'PENDING',
    arrived_at      timestamptz,
    departed_at     timestamptz,
    missed_at       timestamptz,
    minutes_inside  integer,
    out_of_order    boolean          NOT NULL DEFAULT false,
    updated_at      timestamptz      NOT NULL DEFAULT now(),
    CONSTRAINT route_stop_once UNIQUE (plan_id, seq),
    CONSTRAINT route_stop_kind_values CHECK (kind IN ('MANUAL', 'FENCE')),
    CONSTRAINT route_stop_status_values CHECK (status IN ('PENDING', 'INSIDE', 'ARRIVED', 'DEPARTED',
'PASSED', 'MISSED'))
);

CREATE INDEX route_stop_plan ON route_stop (plan_id, seq);
```